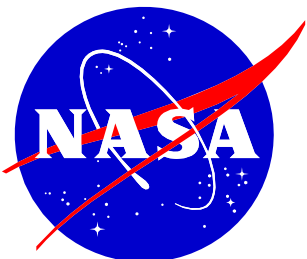


**Fermi GAMMA-RAY
SPACE TELESCOPE
(FERMI)**

**FERMI SCIENCE SUPPORT CENTER
LAT SAE DATABASES DETAILED DESIGN**

DRAFT v0.6

August 05, 2010



**GODDARD SPACE FLIGHT CENTER
GREENBELT, MARYLAND**

http://fermi.gsfc.nasa.gov/ssc/dev/baselined_documents/ for the latest baselined version
and http://fermi.gsfc.nasa.gov/ssc/dev/current_documents/ for the latest draft version

FERMI GAMMA-RAY SPACE TELESCOPE
(FERMI)

FERMI SCIENCE SUPPORT CENTER
LAT SAE DATABASES DETAILED DESIGN

August 05, 2010

NASA Goddard Space Flight Center

Greenbelt, Maryland

Check http://fermi.gsfc.nasa.gov/ssc/dev/baselined_documents/ for the latest baselined version
and http://fermi.gsfc.nasa.gov/ssc/dev/current_documents/ for the latest draft version

**FERMI SCIENCE SUPPORT CENTER
INGEST SYSTEM DETAILED DESIGN**

Prepared by:

Tom Stephens, FSSC Programmer

Date

Concurrence:

Dave Davis, DASS Manager

Date

Ken Lehtonen, Fermi Ground
System/Operations Manager

Date

Approved by:

Chris Shrader, Fermi SSC Manager

Date

CHANGE RECORD PAGE

DOCUMENT TITLE: FERMI Science Support Center			
DOCUMENT DATE: August 05, 2010			
ISSUE	DATE	PAGES AFFECTED	
Original version		All	
Reordered documents sections. Added content to sections 3, 4, and 5.	05/20/05	p. 7 to end	
Added new section 6. Added section headings in section 7. Added content to section 7.1.1 & 7.2. Added new section 1.1. Updated Acronym list. Added STATUS_RESPONSE message (section 5.3.12)	06/08/05	p. 1-2, p. 20 to end	
Added NEW_SEARCH, IDENTIFY, SEARCH_FINISHED, SEARCH_FAILED, and REPROCESSED_DATA_AVAILABLE messages. Added content to Sections 6 and 7.3	06/13/05	p. 20-24, p. 37 to end	
Various minor text updates and clarifications	06/24/05	all	
Content added to the Ingest and MySQL database sections. Various other minor updates	03/07/06	front matter, p. 17, p. 22-23, p. 48 to end	
Change GLAST to Fermi and added new DROP message.	04/08/10	All	
Some updates to try and bring things up to date.	08/05/10	All	

TABLE OF CONTENTS

TABLE OF FIGURES	vii
TABLE OF TABLES	vii
1 Introduction.....	1
1.1 Document Organization.....	1
1.2 Related Documents	1
1.3 Acronyms and Abbreviations	2
2 General Description	3
2.1 Basic Data Flow	4
2.2 User Interface.....	5
2.2.1 Web Interface.....	5
2.2.2 Command Line Interface	6
2.3 QueueManager.....	6
2.4 Database Servers.....	6
2.4.1 Control Node.....	6
2.4.2 Search Node	7
2.5 Ingest.....	7
3 Internal Data Formats	9
3.1 Photon Database.....	9
3.1.1 Standard Region Queries	9
3.1.1.1 Spatial Binning.....	9
3.1.1.2 Temporal Binning	10
3.1.1.3 Chosen Binning Scheme	10
3.1.1.4 Filenames and Directory Structure	10
3.1.2 All-sky Queries	11
3.2 Event/Extended Database	12
3.3 Pointing, Livetime and History Database	12
4 Configuration Files	14
4.1 QueueManager Configuration File	14
4.2 Event/Photon Database Configuration File	16
4.3 Spacecraft Database Configuration File	18
4.4 Ingest Configuration File	20
5 Defined System Messages	21
5.1 Message Structure.....	21
5.2 Message Transport.....	21
5.3 Message Definitions.....	22
5.3.1 NEW_WEB_QUERY message	22
5.3.2 NEW_QUERY message	23
5.3.3 NEW_SEARCH message	23
5.3.4 WEB_QUERY_RECEIVED message.....	24
5.3.5 QUERY_RECEIVED message	24
5.3.6 BAD_QUERY message.....	24
5.3.7 QUERY_FINISHED message.....	25
5.3.8 SEARCH_FINISHED message.....	25
5.3.9 SEARCH_FAILED message.....	25
5.3.10 DISCONNECT message.....	26

5.3.11 DROP message	26
5.3.12 QUERY_STARTED message	26
5.3.13 PROGRAM_CONTROL message	26
5.3.14 STATUS_RESPONSE message	27
5.3.15 IDENTIFY message	27
5.3.16 NEW_DATA_AVAILABLE message	28
5.3.17 REPROCESSED_DATA_AVAILABLE message	28
5.3.18 DATA_ADDITION_COMPLETE message	28
5.3.19 UNAVAILABLE message	29
6 Priority Queues	30
6.1 Standard Query Queue (Q1)	30
6.2 Large Query Queue (Q2)	30
6.3 Batch Query Queue (QB)	30
6.4 Queue Scheduling Algorithm	30
7 Detailed Design	32
7.1 User Interface	32
7.1.1 Web Interface	32
7.1.1.1 Program Flow	32
7.1.1.2 Query ID String Format	35
7.1.1.3 Query Submission Message Format	35
7.1.1.4 LATDataQuery.cgi Script	37
7.1.1.4.1 Query Parameter Input page	37
7.1.1.4.1.1 Link to help	37
7.1.1.4.1.2 Data summary	37
7.1.1.4.1.3 Query parameter fields	37
7.1.1.4.1.3.1 Sky Position	37
7.1.1.4.1.3.2 Time Range	38
7.1.1.4.1.3.3 Energy Range	38
7.1.1.4.1.4 Database choice	38
7.1.1.4.1.5 Form Buttons	38
7.1.1.4.2 Query Submission Status page	39
7.1.1.5 QueryResults.cgi Script	39
7.1.2 Command Line Interface	40
7.2 QueueManager	40
7.2.1 General Program Flow	40
7.2.2 Message Processing	42
7.2.2.1 NEW_WEB_QUERY	42
7.2.2.2 QUERY_RECEIVED	42
7.2.2.3 QUERY_STARTED	42
7.2.2.4 QUERY_FINISHED	43
7.2.2.5 BAD_QUERY	43
7.2.2.6 DISCONNECT	43
7.2.2.7 PROGRAM_CONTROL	43
7.2.2.7.1 Shutdown	43
7.2.2.7.2 Status	44
7.3 Database Server	44

7.3.1 Control Node.....	45
7.3.1.1 General Program Flow.....	45
7.3.1.2 Internal Metadata Tables.....	47
7.3.1.2.1 Time Metadata	47
7.3.1.2.2 File Metadata	47
7.3.1.2.3 Search node status metadata	47
7.3.1.3 Starting a New Query.....	47
7.3.1.4 Message Processing	48
7.3.1.4.1 IDENTIFY	48
7.3.1.4.2 NEW_QUERY.....	48
7.3.1.4.2.1 Time range only queries.....	49
7.3.1.4.2.2 All other queries.....	49
7.3.1.4.3 SEARCH_FINISHED.....	50
7.3.1.4.4 SEARCH_FAILED.....	50
7.3.1.4.5 DISCONNECT	51
7.3.1.4.6 NEW_DATA_AVAILABLE	51
7.3.1.4.7 REPROCESSED_DATA_AVAILABLE.....	51
7.3.1.4.8 PROGRAM_CONTROL.....	51
7.3.1.4.8.1 Shutdown	51
7.3.1.4.8.2 Status.....	52
7.3.2 Search Node	52
7.4 Ingest System.....	54
7.4.1 Program Flow.....	55
7.4.1.1 New Data	55
7.4.1.2 Reprocessed Data.....	55
8 MySQL Database Table Descriptions	56
8.1 Ingest_data Table.....	56
8.2 photon_file_comp Table.....	56
8.3 photon_file_data Table	56
8.4 photon_allsky_file_comp Table.....	57
8.5 photon_allsky_file_data Table.....	57
8.6 photon_query_params Table.....	58
8.7 photon_query_status Table	58

TABLE OF FIGURES

Figure 1 - Basic design of the database system. Arrows show the connections between the various components and the direction of data flow.	3
Figure 2: Dataserver components and their host machines.	4
Figure 3 - Example of data directory structure for photon data. Each of the blue boxes (N0-S3) expands into the 16 green boxes with the first digit in the sub-directory name corresponding to the digit in the name of the higher level directory. Within each sub-directory, there are four files per time interval corresponding to the 4 HTM level 3 nodes stored in that directory.	11
Figure 4 - Structure of a database message transmission. Blue boxes represent data elements of the message and the green boxes are the newline field delimiters.	22
Figure 5 - Program flow for LATDataQuery.cgi script.....	33
Figure 6 - Program flow for QueryResults.cgi script	34
Figure 7 - Basic program flow for the QueueManager.....	41
Figure 8 - Basic program flow for the control node process of the database system	46
Figure 9 - Basic program flow for the database search node process.....	54

TABLE OF TABLES

Table 1 - Parameters for QueueManager configuration file.	14
Table 2 - Parameters for photon/event database configuration file	16
Table 3 - Components of a database message.	21
Table 4 - Commands sent via the PROGRAM_CONTROL message	27
Table 5 - Keywords and values for the query parameter string to be sent to the QueueManager.	36
Table 6 - Contents of the QueueManager portion of the STATUS_RESPONSE message.....	44
Table 7 - Contents of the control node's STATUS_RESPONSE message	52

1 Introduction

This document describes the design of the LAT SAE database systems at the FSSC. It covers the software developed to serve out the LAT Event/Extended data, the LAT Photon data and the LAT Pointing, Livetime and Mode History data. The databases will be referred to throughout the document as the D1ev, D1ph and D2 databases or simply the event, photon and spacecraft databases respectively.

This document, together with the relevant requirements documents, should provide all the information necessary to develop and test the database systems described herein.

This document assumes that the reader is familiar with the contents and nature of the LAT event, photon and spacecraft data sets, as described in the Science Data Products ICD, the requirements on the data system described in the requirement documents (GSSC-0006 & GSSC-0007) as well as the FSSC standard operating environment described in the GSSC Standard Operating Environment Document.

1.1 Document Organization

This document is laid out in the following manner. The remainder of this section list various related documents and gives a list of acronyms and abbreviations used throughout the document. Section 2 gives a short description of the entire data system and briefly describes the interplay between the various components. Section 3 describes the way the data is stored internally within the data system and some of the motivation behind this choice of storage scheme. Section 4 describes the various configuration files used to initialize the data system at start up and gives the details of their contents. Section 5 lists the various internal data messages that are passed between the components of the system, their format and content. Section 6 describes the query priority queues and the algorithm used to schedule the queries in the queues. Section 7 gives a detailed description of each component of the data system and describes the operations of the software, algorithms used and details of the effects of the various messages on each part of the system. Finally, section 8 describes the MySQL data tables that are used by the data system to maintain state and metadata information.

1.2 Related Documents

[GSSC Design Document](#) (GSSC-0003)

[GSSC Test Plan](#) (GSSC-0005)

[LAT Event Summary Database Requirements](#) (GSSC-0006)

[Standard Analysis Environment Database Requirements](#) (GSSC-0007)

[Science Data Products ICD](#)

[GSSC Functional Requirements Document](#) (433-RQMT-0002)

[Ingest System Detailed Design Document](#) (GSSC-0009)

GSSC Standard Operating Environment Document

LAT Science Analysis SAE Design Document

HTM Documentation – <http://skyserver.org/htm>

FITSIO User's Guide – <http://heasarc.gsfc.nasa.gov/docs/software/fitsio/fitsio.html>

1.3 Acronyms and Abbreviations

D1ev	LAT Event/Extended Database
D1ph	LAT Photon Database
D2	LAT Pointing Livetime and Mode History Database.
DASS	Data Archive and Software Support
Dec	Declination
FITS	Flexible Image Transport System
FTP	File Transfer Protocol
GeV	Giga electron Volts = 10^9 eV
Fermi	Fermi Gamma-ray Space Telescope
FSSC	Fermi Science Support Center
HTM	Hierarchical Triangular Mesh
ICD	Interface Control Document
IOC	Instrument Operations Center
LAT	Large Area Telescope
LISOC	LAT Instrument and Science Operations Center
MET	Mission Elapsed Time
MeV	Mega electron Volts = 10^6 eV
MJD	Modified Julian Date
RA	Right Ascension
SAE	Standard Analysis Environment
SDSS	Sloan Digital Sky Survey

2 General Description

The database system to serve out the LAT event, photon and spacecraft data consists of four main components: the User (web) Interface, the QueueManager, the database servers and the ingest system. These systems interact with a MySQL database. The general relationship between the various parts of the system is shown in Figure 1. Each part of the system performs a different function and may consist of several programs. Each is briefly described in the following subsections.

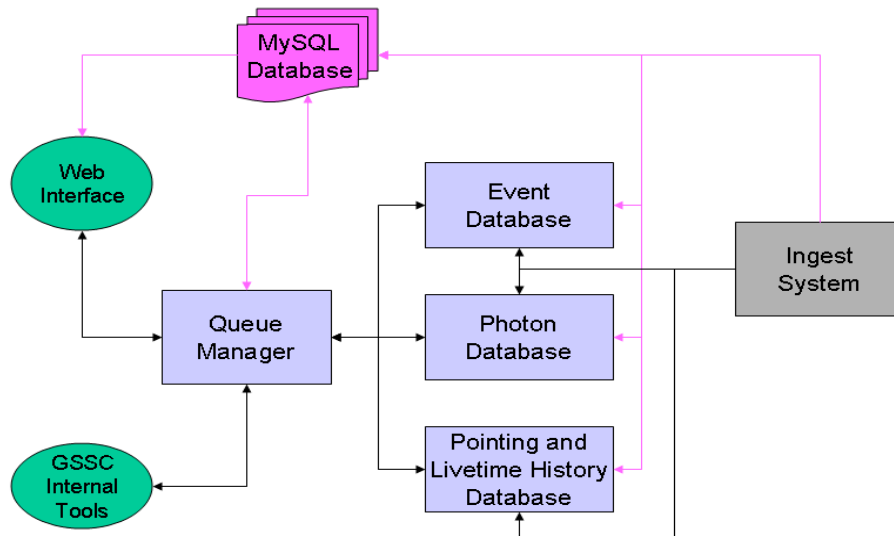


Figure 1 - Basic design of the database system. Arrows show the connections between the various components and the direction of data flow.

The system is designed to operate on a cluster or clusters of workstations communicating by TCP/IP connections in a client/server relationship with defined messages passing back and forth to control the processing and delivery of data to the servers and the user. The system also makes use of a MySQL database to record information about the data in the server as well as the queries that have been requested, are being processed or have been completed.

A more current although slightly busier version of the basic design can be seen in Figure 2. This includes what machine each component generally runs on.

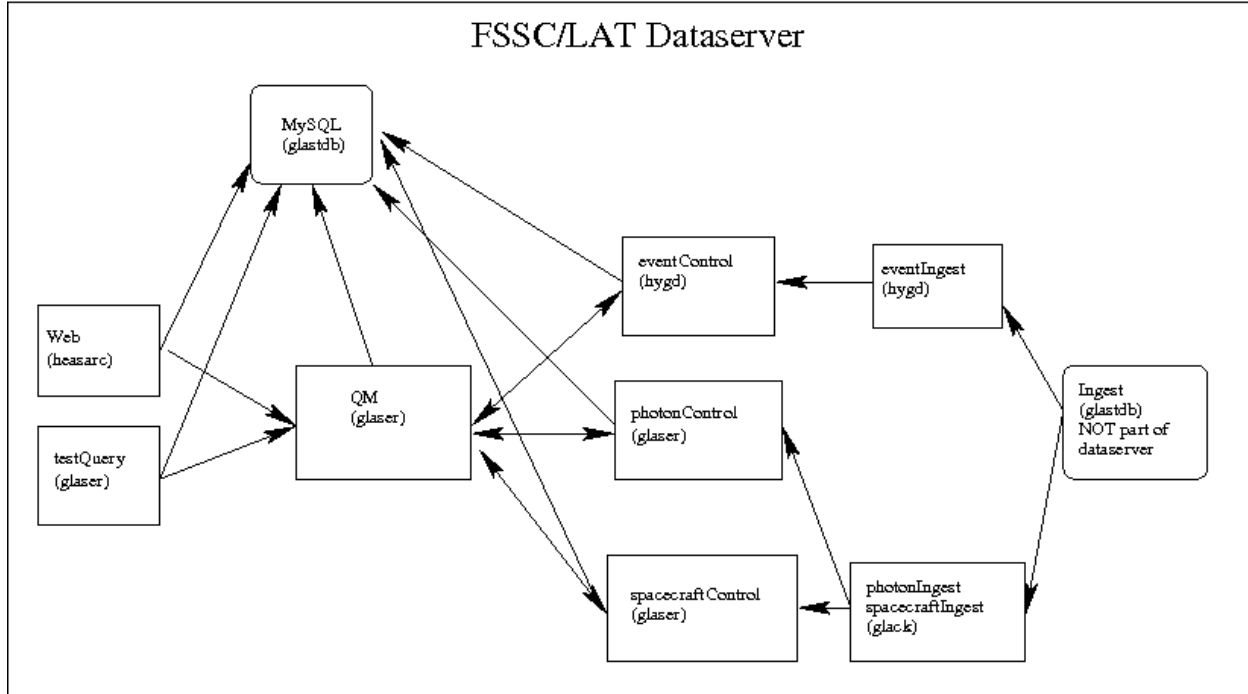


Figure 2: Dataserver components and their host machines. Note: New hardware purchases have the event server processes running on glutton as hygd has been retired.

2.1 Basic Data Flow

A query begins when the user inputs the search criteria on the web interface page. This data is sent to the QueueManager which validates the query information, assigns a priority to the query, and submits it to the relevant database servers. The servers place the query in the appropriate queue based on the priority assigned by the QueueManager. A message is then sent to the QueueManager with the estimated time until the query will be completed.

Once the time estimates are received from all servers involved in the query, the QueueManager notifies web script of the estimated completion time. This causes a new page to be displayed informing the user of the query ID, the estimated time to completion and providing a link to a second script which will display the results of the query. It also provides the option of the user receiving an e-mail notification when the query is complete. (*Note: e-mail notification is currently disabled.*)

When a query reaches the top of its queue, the data is retrieved by the servers and placed on a public FTP site for 3 days. A perl script runs nightly on the dataserver to remove queries older than the expiration time. Currently, after 3 days, queries are no longer visible through the web interface but can still be retrieved through ftp. After 7 days, queries are deleted although this can be shortened if disk space issues force it. The QueueManager is informed that the query is complete and is removed from the list of open queries on both the servers and the QueueManager. If the user requested e-mail notification, the QueueManager will send the e-mail before removing the query from its list.

The second web script checks to see if the query has been completed. If not, it provides the user with an estimate of the remaining time before the completion. If the query has been completed, the script provides the user with a summary of the query parameters and links to the data on the FTP site which they can download.

2.2 User Interface

The user interface has two distinct parts designed for different purposes. The first, a web interface, is the default interface that will be used by the world at large to interact with the

system. The second is a command line executable interface designed to be used by the system maintainers and testers and will not be generally available to the public. The command line interface is not yet fully implemented.

2.2.1 Web Interface

The web interface is the visible portion of the data system. It was modeled to look similar to the HEASARC Browse data archive system and was planned to be linked to from that system. The web interface consists of two scripts written in Perl CGI.

The first script presents a web page allowing the user to input their search parameters and select the database(s) to be queried for the data. The script will allow entry of a source position or name, a time range to perform the search on and an energy range.

When the submit button is clicked, the script does some basic range checking on the data to make sure it is valid. If not a message is given to the user and the data can be corrected. If it is valid, the script establishes a TCP/IP connection to the QueueManager and sends the query data across the connection. This connection is held open until a response is received from the QueueManager.

Once the query data is submitted, the script will receive a message from the QueueManager indicating either that there was a problem with the data and indicating the type of problem or that the query was successfully submitted and indicating the estimated time to query completion. If the query parameters had an error, the nature of the error will be presented to the user and they will have an opportunity to correct the data and resubmit the query.

If there were no errors in the data, the user will be presented with a page that lists the query parameters that were transmitted to the server, and informs the user of the estimated time until the query will be completed. It then provides a link to the second web script that will display the results of the query. Finally, the page presents a small form allowing the user to enter their e-mail address if they wish to be informed by e-mail of the query completion. This may be desirable as some queries, especially those going to the event database, may take many minutes or even hours to complete and the user may not want to sit around that long to check periodically to see if the query is done.

The second script that makes up the web interface is a script designed to present the results of the query to the user. If loaded via the link on the previous page, it checks to see if the specified query has been completed. If it has been completed, the page lists a summary of the query parameters that generated the data and a list of all the files containing the query results. For each file, there is a link to the data on the FTP server and basic statistics such as file size and number of events are presented.

If the query is not yet finished, the page will display a message informing the user of the fact and indicating the estimated remaining time until the query will be done. In both cases, the page will contain a form at the bottom allowing the user to enter the query ID supplied by the first script for a different query to check on its status if desired. (*Note: This seems to be disabled in the new interface.*)

If the script is loaded directly in the browser, the user will be presented with a small form allowing them to enter the query ID of the query that they wish to check the results of. Once the query ID is entered, the script proceeds as if linked to from the query submission script.

2.2.2 Command Line Interface

This interface is designed to be used by developers, testers, and those maintaining the system. It provides all the functionality to submit queries but also allows the user to query about the internal status of the system and shutdown the system if necessary. It is implemented as a compiled, executable program that can be run on any machine with TCP/IP access to the listen port of the QueueManager.

2.3 QueueManager

The QueueManager is the link between the user interface and the database servers that perform the queries. It acts as a TCP/IP server for connection from the user interface and a TCP/IP client for connections to the database servers. Its purpose is to collect queries from the user interface, validate them, assign priorities based on the query parameters and send the queries to the appropriate database servers. It receives messages from the servers indicating the status of queries, such as estimated time to completion and notification of the starting and completion of query processing, and forwards these, where appropriate, to the user interface.

The QueueManager also monitors the connections to the database servers and automatically attempts to re-establish connections that are lost. When there is no connection to a database server, the QueueManager informs the user of the temporary loss of connectivity if a query to that server is submitted.

At its core, the QueueManager is an asynchronous, message driven event engine that triggers on messages received on its many TCP/IP sockets. When a new message is received it is read and processed depending on the message source and type.

Internally, the QueueManager maintains tables of all open TCP/IP socket connections and all outstanding queries, their parameters and their status. As connections are made and closed and queries are received and completed these tables are updated to reflect the current status of the database system as a whole.

2.4 Database Servers

The database servers form the core of the data system and do the work of extracting data from the database in response to queries submitted by the users. The database servers are designed to be operated on a cluster of workstations each running a part of the server system. The servers consist of two parts, the Control node and the Search nodes, each performing a different task. It is envisioned that a database server will run on a homogeneous Beowulf Linux cluster with the Control node process running on the cluster's head node and the Search node processes running on the cluster's compute nodes. In practice, there is a homogeneous cluster for the photon and spacecraft servers with the event/extended data running on single multi-core server with a large allocation of memory and disk space.

There is one database server system for each of the three databases with the difference being the type of data handled and the number of search nodes required to process the data and meet the system requirements.

2.4.1 Control Node

The control node is the interface between the QueueManager and the search nodes. It is responsible for receiving the query requests from the QueueManager, maintaining the query

queues, estimating the query completion time, and scheduling the query processing on the search nodes. It acts as a TCP/IP server for both the QueueManager and the search nodes.

Like the QueueManager, the Control node is an asynchronous, message driven event engine that triggers on messages received on its many TCP/IP sockets. When a new message is received it is read and processed depending on the message source and type.

Internally, the Control node maintains a list of all the internal data files in the data server as well as various metadata about those files. It also maintains a list of all the TCP/IP connections as well as information about the status of the search nodes.

The primary role of the Control node is to schedule the processing of queries on the search nodes in an optimal manner. Based on the number of free search nodes, the contents of the various priority queues and the parameters of the query to be processed, the control node determines the number of search nodes to be applied to the current query. The relevant list of internal data files to be searched is then divided among the search nodes and messages are sent to each node indicating the files to be searched and the query parameters. Once all the search nodes have finished their work, the control node sends a message to the QueueManager indicating that the query is complete.

2.4.2 Search Node

The search node process is the workhorse of the database system that actually performs the query. It's only function is to perform the data searches assigned by the Control node. It acts as TCP/IP client to the Control node and establishes a connection upon startup that is maintained until the search node shuts down.

Upon startup the search node will send a message to the Control node indicating that it is ready to process data and then waits for a return message. When a query is to be performed, the search node will receive from the Control node a message containing a list of files and the query parameters for the search. It then searches those files for data that matches the query, storing the data in an output data file. Once all the data files have been searched, the output file is closed and moved to the FTP site and a message is sent to the Control node indicating that the query is complete and that the search node is ready for a new task.

2.5 Ingest

Data ingest into the database system is handled by specialized software that takes the Level 1 data received from the LISOC and converts it into the internal data format needed by the database system. This software also informs the database system of the availability of new data so that the database system can upload the new data into its internal files and update the metadata. The details of the internal data formats are given in Section 3.

While the internal data format for each of the three databases is different, the ingest process is similar. First the ingest program checks to see if the relevant Control process is up and responding. If it is not, the ingest does not happen and an error page is sent. If the Control process is up, the Ingest process reads in the data received from the LISOC and divides it into the appropriate time and/or spatial bins for the specific database's internal format. The ingest process then appends the new data to the data already in the database. Once this is done, the database server is informed of the availability of the new data and adds it to the internal data set.

The ingest system also maintains a backup copy of the internal metadata used by the server in a MySQL database. This copy is used to restore the metadata to the servers in the event of a crash. NOTE: The data kept by the ingest system (external to the dataserver) cannot be used

to rebuild any MySQL data for the dataserver. In practice, the MySQL data is never the problem anyway. A system to make sure the file data and is in sync with the MySQL data (which is the more common problem) needs to be devised.

3 Internal Data Formats

The data for all three databases are stored internally as FITS files. The internal files have the same data format as the files returned to the user via a query but are organized in such a manner to optimize search speed for the typical types of searches to be performed on the data in the specific database. The detail of each database's internal representation are given below

The data in the database will be constantly growing during the mission with 6-8 data updates planned each day.

3.1 *Photon Database*

As this is the primary database for the Fermi mission and will be the most heavily used, the internal data files have been heavily optimized to provide the best speed when doing searches on this database. As the data volume for this database is not too large, the data is actually stored internally in two different formats to optimize for different types of queries.

3.1.1 Standard Region Queries

It is expected that the most common type of query will be a query on a specific region of the sky for some time period, possibly the entire length of the Fermi mission. Therefore the database system and its internal data representation have been optimized for this type of search.

As the entire database system works by reading and writing FITS files, the amount of time required to complete a search is directly related to the total data volume that must be scanned to find the photons matching the query parameters. Thus to reduce the query time we need to reduce the amount of data that must be searched to fulfill the request. To this end, the internal data for the photon database is divided into spatial bins with each data file containing all the events in that bin for some amount of time.

3.1.1.1 Spatial Binning

By dividing the sky into spatial bins, we only need to look at events that fall in the bins that overlap the queried sky position. Due to the nature of the LAT data, a "standard" query will request data from a region of the sky described by a circle with a radius of 15° or a box $30^\circ \times 30^\circ$ on a side. An area this size represents $\sim 5\%$ of the entire sky. Thus, in order to search efficiently we want our bin size to be somewhat smaller than that size.

In order to divide the sky into spatial bins, several binning methods were considered. In the end we chose to use the Hierarchical Triangular Mesh (HTM) tessellation system for our spatial binning scheme. HTM was originally developed for use with the Sloan Digital Sky Survey at the Johns Hopkins University.

This system divides the sky in the following manner. The sky is initially divided into 8 spherical triangles with vertices at the celestial poles and the 0, 90, 180, and 270 degree points on the celestial equator. This is considered level 0. Each successive level of tessellation divides the spherical triangles from the level before into four smaller triangles by connecting the centers of the sides of the larger triangles. The triangles are not all exactly the same size but are roughly equal. Each level has 8×4^n triangles where n is the level of tessellation.

3.1.1.2 Temporal Binning

The time ranges for a query are expected to be quite varied although typically we expect them to be relatively long, i.e. on the order of months or years. Thus we don't want to put all the data for a given spatial bin into a single file, as we would then be sifting through a large amount of unnecessary data for queries with short time ranges. On the other hand we also don't want to have lots of small files lying around on the disk that need to be tracked and searched as opening and closing the individual data files has a performance overhead as well as a data storage overhead due to the size of the FITS headers for the files.

3.1.1.3 Chosen Binning Scheme

With the choice of HTM for the tessellation algorithm, tests were conducted to find the optimum combination of spatial and temporal bin sizes. The figure of merit used to optimize the bin size was the total time to complete a query requesting one year of data for a circular region with a 15° radius. This is one of the two "standard" queries defined in the [LAT Event Summary Database Requirements](#) document. A grid of various HTM levels from 2 to 6 and various time bins ranging from 1 day to 1 year were used to compute the average query time for a sample of 100 queries.

Based on these tests, it was determined that the optimal bin size was to use HTM level 3 with two month time bins. This resulted in an average query time of ~40s. HTM level 3 divides the sky into 512 triangles each covering approximately 80 sq degrees of sky and a typical "standard" query involves searching 12-20 of these triangles.

3.1.1.4 Filenames and Directory Structure

The filenames for these internal pixilated data files have the following format:

`<server name>_tsNNN_HXXXX_VXX.fits`

where `<server name>` is either photon or event depending on the server (the spacecraft data has not concept of spatial binning), `tsNNN` is the internal time interval, `HXXXX` is the HTM pixel ID and `VXX` is the version number of the file. The version number is only updated when reprocessed data is received that replaces data already in the file. (Note: Handling of version number needs to be thought out a little more. This is the current scheme but will probably change. And in fact, versioning has never actually been implemented.)

Implementation note: The <server name> value should be changed to be just a two letter value (EV or PH) instead of 'event' or 'photon' as currently implemented in order to simplify some of the internal workings of the data server code.

The data is stored in a directory tree under the root data directory. This tree consists of two additional levels of directories below the root directory as shown in Figure 3. Under the root directory there are 8 subdirectories corresponding to the 8 primary triangles in the HTM tessellation scheme. Within each of those directories, there are 16 subdirectories corresponding to the 16 HTM level 2 triangles within that primary triangle. Within these directories are stored the individual data files corresponding to the four HTM level 3 triangles within the specified level 2 triangle. Thus for every time bin there will be four files within each directory for a total of ~240 files in each directory over the 10 year expected life of the Fermi mission.

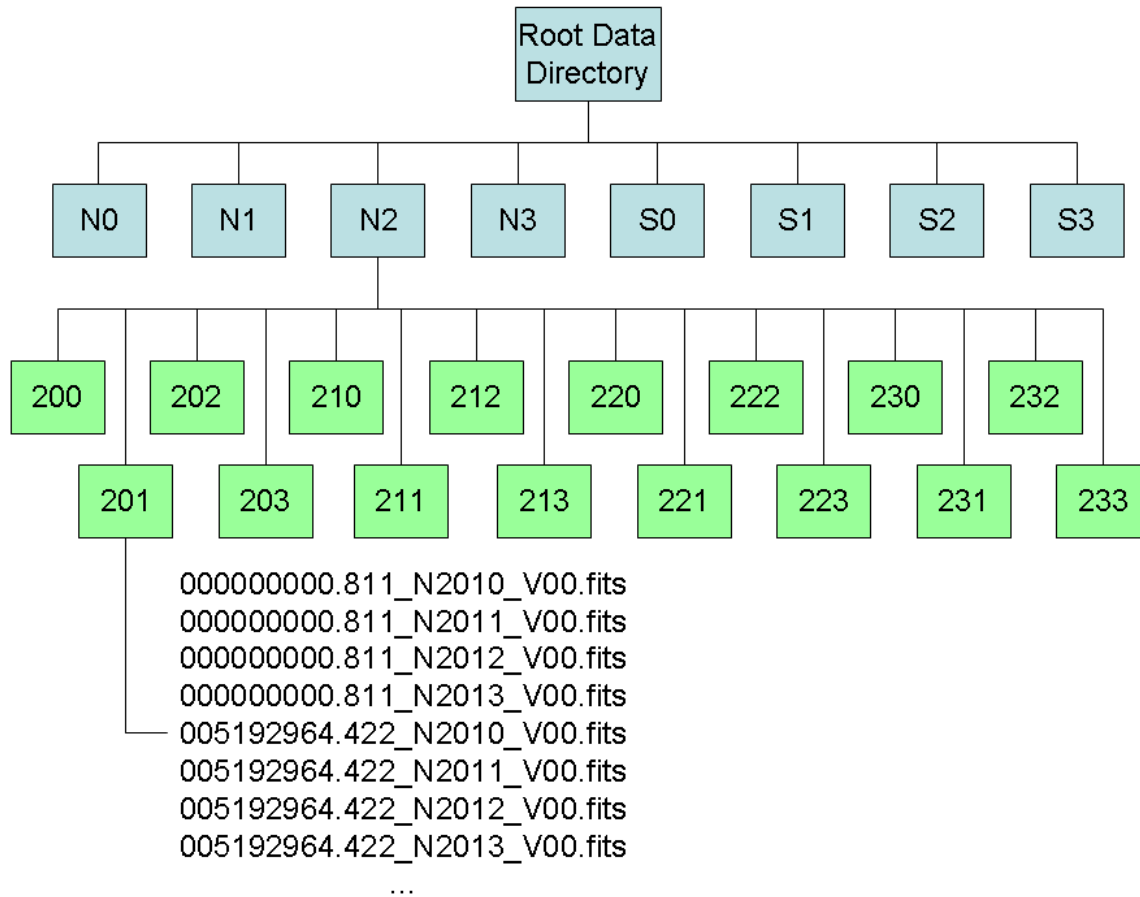


Figure 3 - Example of data directory structure for photon data. Each of the blue boxes (N0-S3) expands into the 16 green boxes with the first digit in the sub-directory name corresponding to the digit in the name of the higher level directory. Within each sub-directory, there are four files per time interval corresponding to the 4 HTM level 3 nodes stored in that directory. Note: *File names shown in the diagram correspond to the old naming convention.*

3.1.2 All-sky Queries

All-sky queries are our worst case in terms of performance and frequently would not ever complete. To prevent this, the front end will block any query that encompasses more than a 60° circumference. In those cases, the front end will provide the user with a link to the weekly allsky files which have been pre-generated by the ingest process for the photon data. For spacecraft data, spatial searches don't mean anything so this limitation does not matter.

Currently, there are no allsky files provided for event/extended data. This was due to the file sizes initially projected for that dataset.

The allsky files generated by the ingest system are stored in the allsky directory under the top data directory. The files have names of the format:

```
lat_photon_weekly_wNNN_pNNN_vNNN.fits
```

where wNNN is the mission week number that the data covers, pNNN is the data processing version from the LISOC for the data and vNNN is the file version number. (*Note: Currently version numbering is not yet implemented.*)

3.2 Event/Extended Database

The file format and data storage matches that of the photon data with a few exceptions. As discussed above, periodic allsky files are not generated for this dataset. Also, unlike the photon data, there is only one copy of the full dataset that is maintained on the event server computer (glutton). There are no redundant copies and the data ingest and searches are all performed locally on the single copy. This was deemed necessary due to the large size of the dataset. This setup slows query times by a factor of a few (but improves ingest times by a similar factor). Since the demand for this dataset is so low (~0.5% of all queries search this database), there are no current plans or need to change this.

3.3 Pointing, Livetime and History Database

Of the three, this database has the simplest internal structure as the data is simply a time ordered list and does not need to be searched spatially. Like the photon database, the internal files for this database are divided into approximately two month time bins with the time boundaries being the same as the time boundaries for the photon database's internal files. This results in an average file size of approximately 16MB for each internal file.

These files will have filenames of the format:

```
lat_spacecraft_weekly_wNNN_pNNN_vNNN.fits
```

where wNNN, pNNN and vNNN have the same meaning as for the photon allsky files. The version number is updated every time new data is added to the file and will typically have a value in the 3-400's for a file covering its full time interval. (*Note: Currently version numbering is not yet implemented.*)

These files will be stored in a single data directory, labeled 'weekly', under the head data directory on the server.

4 Configuration Files

The various servers and programs are written in order to read configuration information from either a default or specified configuration file. Thus several instances of the same program could run with different configurations and not conflict with each other. The configuration files control various aspects of program operation such as location of data directories, TCP ports to listen and talk to, location of MySQL database server, etc.

The configuration files consist of a list of parameters, one per line, in the format of 'PARAMETER value'. Lines beginning with a hash (#) are considered comments. The details and list of parameters for the various configuration files are given in the following subsections.

4.1 QueueManager Configuration File

This file controls the operation of the QueueManager. It gives the names of the computers running the various database server and the ports they are listening on. It also gives the location of the MySQL server that contains the metadata for the queries to be processed.

Table 1 lists the parameters and what they refer to.

Table 1 - Parameters for QueueManager configuration file.

Parameter	Description
LOG_DIR	The directory where the log files for the QueueManager should be stored. Typically this should be \$GSSC_LOG_DIR/Database
EVENT_HOST	The host name of the computer running the Event Database control node
EVENT_PORT	The port number the Event database is listening on for connections
PHOTON_HOST	The host name of the computer running the Photon Database control node
PHOTON_PORT	The port number the Photon database is listening on for connections
SPACECRAFT_HOST	The host name of the computer running the Spacecraft Database control node
SPACECRAFT_PORT	The port number the Spacecraft database is listening on for connections
LISTEN_PORT	The port the QueueManager should use to listen on for connections from the web interface and command line tools.
MYSQL_HOST	The host name of the computer running the MySQL database server
MYSQL_PORT	The TCP port the MySQL database is listening on
MYSQL_DATABASE	Name of the database to use
DB_TYPE	What dataserver component this is (QueueManager)
DB_USERNAME	Name to connect to MySQL server
DB_PASSWORD	Password for MySQL server
PAGE_SYSTEM	Identity for pages (Data_Server)
ENABLE_PAGING	Should pages be sent in the event of errors.
PAGING_PASSWORD	The MySQL password for the paging system

A sample QueueManager configuration file would look like this:

```
# Configuration file for the GSSC Database QueueManager
# All lines containing a # as the first character are comments
```

```

# Values are of the form 'KEYWORD value'
# Any entries after the value are ignored
LOG_DIR           /data/glaser/Pass6/log/
EVENT_HOST        hygd
EVENT_PORT        45289
PHOTON_HOST       localhost
PHOTON_PORT       45288
SPACECRAFT_HOST   localhost
SPACECRAFT_PORT   45290
LISTEN_PORT       45292
MYSQL_SERVER      glastdb
MYSQL_PORT        3307
MYSQL_DATABASE    SAE_DB_Metadata_P6
DB_TYPE           QueueManager
DB_USERNAME       gsscops
DB_PASSWORD       gsscops
PAGE_SYSTEM       Data_Server
ENABLE_PAGING     true
PAGING_PASSWORD   password

```

4.2 Event/Photon Database Configuration File

This file controls the operation of the Photon or Event database. It provides the base directories for the data and log files as well as various parameters governing the way the data is organized internally. It also gives the location of the MySQL server that contains the metadata for the queries to be processed and has a parameter that tells the software which database it is (i.e. photon or event). Table 2 lists the parameters and what they refer to.

Table 2 - Parameters for photon/event database configuration file

Parameter	Description
HOME_DIR	The base directory for the cluster search nodes data directory.
FTP_HOST	The host name of the computer where the FTP disk is located
URL_PREFIX	The URL prefix for the FTP site to be prepended to the file names of the output files
FTP_DIR	The directory on the FTP_HOST where the files should be located
INGEST_DIR	The directory where the ingest program for this database keeps its local copy of the current files
NEW_DATA_DIR	The directory the ingest program places the new data files in to be added to the system.
LOG_DIR	The location where log files should be stored. Typically this will be \$GSSC_LOG_DIR/Database
ARCHIVE_DIR	The location where the archive copies of the files should be stored. (Not currently in use)
HEAD_DATA_DIR	The directory where the control node stores its copy of the data.
SOURCE_DIR	Location of the input files from the LISOC
SERVER_PORT	The port number the control node should listen on for

	connections from the search nodes, QueueManager and the Ingest program
HTM_LEVEL	The HTM level used for the internal data. The default is level 3.
MYSQL_SERVER	The host name of the computer running the MySQL server.
MYSQL_PORT	The port number on the MYSQL_SERVER to connect to. Typically this will be \$GSSC MYSQL_PORT
MYSQL_DATABASE	The name of the MySQL database that holds the metadata for the database system
DB_TYPE	The type of database the software should operate as. Choices are 'photon' or 'event'
TIME_BIN_SIZE	The number of mission weeks that the ingest system should allow to accumulate in a HTM pixel data file before creating a new one.
ALLSKY_TIME_BIN_SIZE	Number of seconds to include in an allsky file before starting a new one. This parameter is depreciated and ignored as the all sky files now correspond to the mission weeks.
CONTROL_NODE	Name of machine Control process will run on
DB_USERNAME	Name to connect to MySQL server
DB_PASSWORD	Password for MySQL server
PAGE_SYSTEM	Identity for pages (Data_Server)
ENABLE_PAGING	Should pages be sent in the event of errors.
PROC_VER	Processing version of the dataset. This should allow multiple, different datasets to be ingested at the same time.
BULK_INGEST	(Optional parameter) Flag for whether or not to use the bulk ingest mode that bypasses checking for the presence of the control process and just does an ingest locally. Default is false.
PAGING_PASSWORD	Password for the paging system to connect to the MySQL database
MIN_INT_PER_NODE	(Optional parameter) Minimum number of time intervals to send to a search node. The default if not included is two (4 months of data).

A sample photon database configuration file would look like this:

```
#####
# Sample configuration file
# These are the settings used when testing on hygd.
# Any line that begins with a '#' character is a comment and is ignored.
# Edit the second column to reflect your specific site configuration.
# The order of the keywords does not matter but spelling does.
#####
HOME_DIR           /data/nodes/Pass6/data
FTP_HOST           localhost
URL_PREFIX         http://fermi.gsfc.nasa.gov/FTP/fermi/data/lat/
URL_BASE           http://fermi.gsfc.nasa.gov/cgi-bin/ssc/LAT/
FTP_DIR            /data/glaser/Pass6/ftp
FTP_PATH           /FTP/glast/data/lat
```

```

INGEST_DIR      /data/glack/Pass6/ingest
NEW_DATA_DIR    /data/glaser/Pass6/new
LOG_DIR         /data/glaser/Pass6/log
ARCHIVE_DIR     /data/glaser/Pass6/archive
HEAD_DATA_DIR   /data/glaser/Pass6/head
SOURCE_DIR      /data/lat/phsum
SERVER_PORT     45288
HTM_LEVEL       3
MYSQL_SERVER    glastdb
MYSQL_PORT      3307
MYSQL_DATABASE  SAE_DB_Metadata_P6
DB_TYPE         photon
TIME_BIN_SIZE   8
CONTROL_NODE     glaser
DB_USERNAME     gsscops
DB_PASSWORD     gsscops
PAGE_SYSTEM     Data_Server
ENABLE_PAGING   true
PROC_VER        120
BULK_INGEST     false
PAGING_PASSWORD password
MIN_INT_PER_NODE 6

```

4.3 *Spacecraft Database Configuration File*

The overall format is very similar to the photon configuration file. A sample is given below. All values have the same meaning as in the photon configuration file.

```

#####
# $Id: spacecraftDatabase.config,v 1.9 2010/06/16 15:14:12 vernaleo Exp $
# Any line that begins with a '#' character is a comment and is ignored.
# Edit the second column to reflect your specific site configuration.
# The order of the keywords does not matter but spelling does.
#####
HOME_DIR        /data/nodes/Pass6/data
FTP_HOST        localhost
URL_PREFIX      http://fermi.gsfc.nasa.gov/FTP/fermi/data/lat/
FTP_DIR         /data/glaser/Pass6/ftp
FTP_PATH        /FTP/glast/data/lat
INGEST_DIR      /data/glack/Pass6/ingest
NEW_DATA_DIR    /data/glaser/Pass6/new
LOG_DIR         /data/glaser/Pass6/log
ARCHIVE_DIR     /data/glaser/Pass6/archive
HEAD_DATA_DIR   /data/glaser/Pass6/head
SOURCE_DIR      /data/lat/pt
SERVER_PORT     45290
HTM_LEVEL       3
MYSQL_SERVER    glastdb

```


MYSQL_PORT	3307
MYSQL_DATABASE	SAE_DB_Metadata_P6
DB_TYPE	spacecraft
TIME_BIN_SIZE	5184000.0
ALLSKY_TIME_BIN_SIZE	604800.0
CONTROL_NODE	glaser
DB_USERNAME	gsscops
DB_PASSWORD	gsscops
PAGE_SYSTEM	Data_Server
ENABLE_PAGING	true
PROC_VER	none

4.4 Ingest Configuration File

The ingest system uses the same configuration file as the server it is ingesting data for. Thus it parses the same configuration files as presented in sections 4.2 and 4.3.

5 Defined System Messages

In order for the various systems to communicate with one another various messages must be passed between the various components. This section describes the method of message transfer and the various messages and their content.

5.1 Message Structure

The messages themselves consist of a short header followed by the body of the message. The header contains three elements giving the message type, an identifying string and a message size in number of lines. The body is a series of strings containing the message data. The components of the message are described in Table 3.

Table 3 - Components of a database message.

Component	Data Type	Description
Message ID	Integer	The message ID value signifying what type of message is being sent/received. This allows the receiving system to process the message properly. Always transmitted in network byte order
ID String	String	A string identifying the query that the message is associated with. The query ID is generated by the web script or command line tool and should be propagated throughout the system. This is used to look up data in metadata tables and pull information from internal data structures related to the query. If the message is not specifically associated with a query (such as a server status request) an appropriate identifier should be used.
Message Size	Integer	The number of array elements in the following message body. Always transmitted in network byte order.
Message Body	Array of strings	This is a free form section that varies from message to message containing any additional information needed by the receiving program to process the message. Any numbers to be transmitted will be encoded into strings with the required precision.

5.2 Message Transport

The messages are transferred via TCP/IP socket connections between the various programs and scripts. Each program operates as either a server or client for establishing connections to the other elements.

The transmission of the message is prefaced by sending an integer, in network byte order, specifying the total number of bytes to be transmitted in the following message. The message is then transmitted with each element of the message separated by newline characters (0x10). The individual lines of the message body are also separated by newline characters when transmitted. The number of bytes transmitted in the preface value should include the newline characters in the count.

Figure 4 shows the layout of the message transmission scheme. The blue elements are the message components. The green are the newline delimiters. Also indicated is the size of the fields.

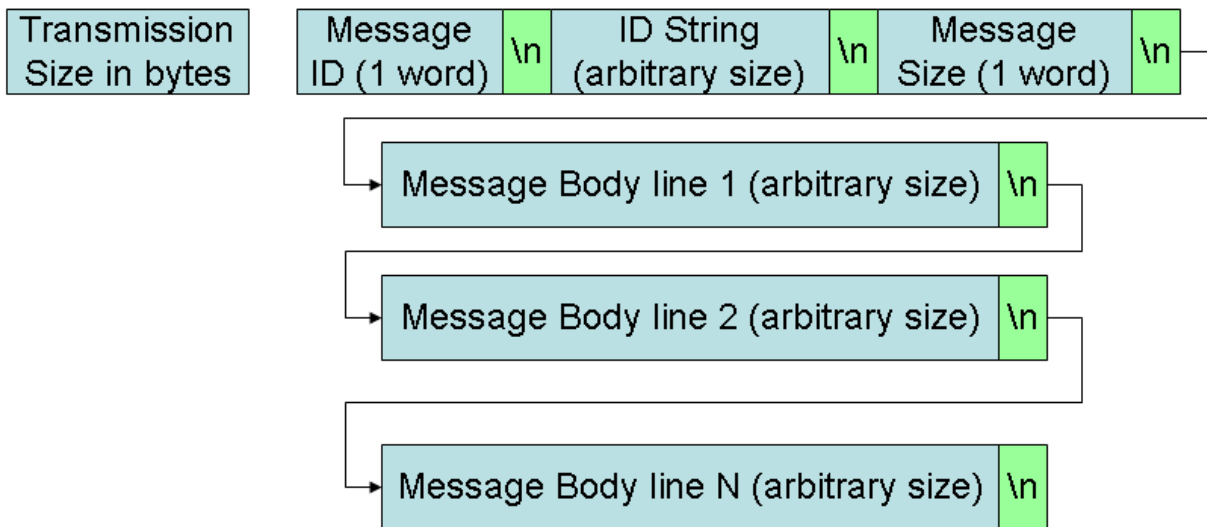


Figure 4 - Structure of a database message transmission. Blue boxes represent data elements of the message and the green boxes are the newline field delimiters.

5.3 Message Definitions

This section gives a description of each of the messages used by the database systems. For each message its ID value and alternate name is given along with a description of what the message is to be used for. Also listed are the programs that expect to send or receive each message.

5.3.1 NEW_WEB_QUERY message

Name: NEW_WEB_QUERY

ID value: 1001

Senders: Web Interface, Command line tools

Receivers: QueueManager

Has Body: yes

Description: This is the first message sent to start a new query. It is initially generated by either the Web Interface or the Command line utilities and is sent to the QueueManager. The QueueManager reads the message and validates the query parameters. If the query passes validation, a WEB_QUERY_RECIEVED message is sent back to the originator of the query to acknowledge the receipt of the valid query.

The QueueManager then determines which the priority queue the query will be in and composes a NEW_QUERY message which is sent on to all the servers involved in the query.

The body of this message consists of two lines. The first is the query parameters in the form a query string as described in section 7.1.1.3. The second is an identifier indicating the source of the query.

5.3.2 NEW_QUERY message

Name: NEW_QUERY

ID value: 1002

Senders: QueueManager

Receivers: Control Node

Has Body: yes

Description:

This message is generated by the QueueManager after the priority queue for the query has been determined. It is sent to the control node of all the servers that will participate in the query.

When this message is received by the control node for one of the servers, the priority queue identifier is extracted from the message and the query is added to that queue. The control node then parses the query parameters and makes an estimate of the total time needed to complete the query. This estimate is then returned to the QueueManager as part of the QUERY_RECEIVED message.

The body of this message consists of three lines. The first two are the same as the body of the NEW_WEB_QUERY message, namely the query parameters and the source. The third line is the priority queue identifier.

5.3.3 NEW_SEARCH message

Name: NEW_SEARCH

ID value: 1003

Senders: Control Node

Receivers: Search Node

Has Body: yes

Description: This is the message sent to the search node by the control node to tell it to start processing a query. The body consists of three parts. The first part consists of a single line that contains the search node ID number. This ID number is used in constructing the output data file names. The ID numbers start at 0 and are incremented for each additional search node used to process the query.

The second part also consists of a single line and contains the query string that the search node will use to create the FITS row filter needed to extract the events and update FITS header keywords.

The final part consists of a list of files which the search node will extract data from. Each line in the list contains a single file. The first character on each line will be either a 'P' or 'F' indicating that the file is either partially or fully enclosed by the search area respectively. This will be followed by a colon and then the path and filename of the file with the path being the relative path below the HOME_DIR directory specified in the configuration file. Thus a typical entry may look like this 'P:N0/000/photon_ts013_N0001_V01.fits'.

5.3.4 WEB_QUERY_RECEIVED message

Name: WEB_QUERY_RECEIVED

ID value: 2001

Senders: QueueManager

Receivers: Web Interface

Has Body: no

Description: This message is sent to indicate that the query parameters were properly received and validated by the QueueManager

5.3.5 QUERY_RECEIVED message

Name: QUERY_RECEIVED

ID value: 2002

Senders: Control Node, QueueManager

Receivers: QueueManager, Web Interface, Command line tools

Has Body: yes

Description: This message is sent from the control node of the database server to the Queue Manager after the query has been received and an estimate of the completion time has been made. Once the queue manager has received this message from all the servers, it generates a QUERY_RECEIVED message to send to the source of the original query.

When the original query source receives this message, it extracts the estimated query time from the message body and informs the user of how long to expect to wait for the search to be complete.

If the message is being sent from the database control nodes to the QueueManager, it will contain a single line in the body. If it is being sent from the QueueManager to the user interface tool, the body of this message will consist of one line per server involved in the query. Each line will contain the name of the server, the priority queue the query is in, the position in the queue and the time, in seconds, until the server estimates the query will be finished. *(This functionality is not yet implemented. Currently just the longest time estimate is returned to the original query source.)*

5.3.6 BAD_QUERY message

Name: BAD_QUERY

ID value: 2003

Senders: Control Node, QueueManager

Receivers: QueueManager, Web Interface, Command line tools

Has Body: yes

Description: This message is sent when some aspect of the query validation process doesn't pass and the query cannot be processed. If sent from the control node to the QueueManager, the QueueManager will forward it to the source of the query request. It is also sent from the QueueManager to the web interface or command line tool when a connection to a database server needed to complete the query is down and the query cannot be processed.

The body of the message is a single line describing what the source of the problem was.

5.3.7 QUERY_FINISHED message

Name: QUERY_FINISHED

ID value: 2004

Senders: Control Node, QueueManager

Receivers: QueueManager, Web Interface, Command line tools

Has Body: yes

Description: This message is initiated by the control node of a database server. It is sent when the processing of a query has been completed. The QueueManager collects these messages for all the servers involved in the query. Once it has received all of them, it closes out the query and removes it from its internal lists.

In the case of all-sky searches, the connection to the query source is still open when this message is received and so the message is forwarded to indicate that the files are ready for download.

The body of this message contains the URL's to the data files containing the query results with one file per line in the body.

5.3.8 SEARCH_FINISHED message

Name: SEARCH_FINISHED

ID value: 2005

Senders: Search Node

Receivers: Control Node

Has Body: yes

Description: This message is sent from the search node to the control node when it is done working on the query specified in the messages ID string. The body contains the name of the output data file created by the node.

5.3.9 SEARCH_FAILED message

Name: SEARCH_FAILED

ID value: 2006

Senders: Search Node

Receivers: Control Node

Has Body: yes

Description: This message is sent when there search node has a problem processing the query. The body of the message contains a description of the error.

5.3.10 DISCONNECT message**Name:** DISCONNECT**ID value:** 3001**Senders:** All**Receivers:** All**Has Body:** no

Description: This message is sent just prior to closing a TCP/IP socket connection by the program initiating the closure. It signals to the receiver that the socket the message was received on will no longer be valid and should be closed on the receiver's end as well. The Control processes contain some logic for dealing with the number of remaining nodes when dealing with this message so only Search processes should send this to the Control node. For a general disconnect, use the DROP message.

5.3.11 DROP message**Name:** DROP**ID value:** 3011**Senders:** All**Receivers:** Control Process**Has Body:** no

Description: This message handles deleting a socket connection in a more general way than the DISCONNECT message (it follows the original purpose of the DISCONNECT message).

This message is deprecated and does not need to be used. The DISCONNECT message should be used in all cases.

5.3.12 QUERY_STARTED message**Name:** QUERY_STARTED**ID value:** 3002**Senders:** Control Node**Receivers:** QueueManager**Has Body:** no

Description: This message is sent to the QueueManager to inform that the processing of the query has begun.

5.3.13 PROGRAM_CONTROL message**Name:** PROGRAM_CONTROL**ID value:** 3003**Senders:** Command Line tools, QueueManager, Control Node**Receivers:** QueueManager, Control Node, Search Node**Has Body:** yes

Description: This message is sent when some sort of system command needs to be executed by the database system. Typically, it will originate from the command line interface with a request

for server statistics, server status, or a request to shut down the system. It is forwarded throughout the system to the relevant components.

The body of this message consists of a single line with the command that is to be executed. The choice of commands and their effects are listed in Table 4.

Table 4 - Commands sent via the PROGRAM CONTROL message

Command	Description
SHUTDOWN	When this command is received the listen ports are closed, and the programs are marked to terminate. As soon as the current query is completed the system will gracefully terminate all processes. This is to be used to shut down the system during testing and in the event of a known shut down of the facilities (i.e. due to a power failure, system upgrade, etc.)
STATUS	The command is a query for the status of the database systems and should return information on the up-time of the servers, number of queries served, status of the query queues, current queries being processed, etc. (This functionality has not yet been implemented.)

5.3.14 STATUS_RESPONSE message

Name: STATUS_RESPONSE

ID value: 3004

Senders: Control Node, QueueManager

Receivers: QueueManager, Command line tool

Has Body: yes

Description: The body of this message contains the status information on the state of the database system. This message is generated as a response to the PROGRAM_CONTROL message with the STATUS command. It will contain information about the live time of the system, number of queries processed, average query time, etc. The exact details of the message body depend on the source and are described in sections 7.2.2.7.2 and 7.3.1.4.8.2.

5.3.15 IDENTIFY message

Name: IDENTIFY

ID value: 3005

Senders: All

Receivers: All

Has Body: possibly

Description: This message is always the first one sent after a new connection is established. It has no body and the ID string field in the message header will contain the name of the program transmitting the message (i.e. QueueManager, database control node, search node, etc.) The allowed values for these ID's are listed below and are fairly self-explanatory:

- WebInterface
- CommandLine
- QueueManager
- photonControl

- eventControl
- spacecraftControl
- photonSearch
- eventSearch
- spacecraftSearch
- photonIngest
- eventIngest
- spacecraftIngest

If any of the ‘<database>Search’ ID’s are received, the message body will have a single entry containing the name of the node that is functioning as the search node. Otherwise the message body will be empty.

5.3.16 NEW_DATA_AVAILABLE message

Name: NEW_DATA_AVAILABLE

ID value: 4001

Senders: Ingest, Control Node

Receivers: Control Node, Search Node

Has Body: yes

Description: This message is initially sent from the ingest program to the control node of the database server to indicate that a new data set is available to be added to the system. Once received, the control node should read the data from the INGEST_DIR (see section 4.2) into its local data store. Once this is complete, the message is then forwarded to the search nodes, who in turn add the data to their local store.

The message ID string for this message contains an identifier signifying the source of the ingest request message and the start time of the new data being added.

The message body contains a list of filenames that are new and need to be added to the database. The filenames contain the path relative to the INGEST_DIR directory as well as the full filename.

5.3.17 REPROCESSED_DATA_AVAILABLE message

Name: REPROCESSED_DATA_AVAILABLE

ID value: 4002

Senders: Ingest, Control Node

Receivers: Control Node, Search Node

Has Body: yes

Description: This message is not used.

5.3.18 DATA_ADDITION_COMPLETE message

Name: DATA_ADDITION_COMPLETE

ID value: 4003

Senders: Control Node, Search Node

Receivers: Control Node, Ingest, QueueManager

Has Body: no

Description: This message is sent when the program that received the NEW_DATA_AVAILABLE or REPROCESSED_DATA_AVAILABLE message has completed the ingest process and is ready to work with the new data. It has no body and the message ID is the same as the ID in the NEW_DATA_AVAILABLE or REPROCESSED_DATA_AVAILABLE message. It is sent to the QueueManager to inform it that the sending server can now again receive queries.

5.3.19 UNAVAILABLE message

Name: UNAVAILABLE

ID value: 4004

Senders: Control Node

Receivers: QueueManager

Has Body: Yes

Description: This message is sent to the QueueManager when the Control Node has received the NEW_DATA_AVAILABLE or REPROCESSED_DATA_AVAILABLE message to inform the QueueManager that the server is currently unavailable for requests because it is ingesting data. The body consists of a single line containing a message indicating why the server is unavailable.

6 Priority Queues

The database system defines three different priority queues. The first queue, Q1, is for “standard” queries. The second, Q2, is for “large” queries that will take somewhat longer than standard queries and the last queue, QB, is for batch queries or queries from command line tools. This section provides the details on how the queries are assigned to the various queues and how the queries in the queues will be processed.

6.1 Standard Query Queue (Q1)

This queue is for processing all queries that fall under the “standard” search definitions in the [LAT Event Summary Database Requirements](#) Document (GSSC-0006) or queries that are smaller than this defined size. Typically, these will be region searches covering less than 700-900 square degrees of the sky depending on the shape of the region.

This queue has the highest priority and will be allocated the largest number of processors on the computing cluster in order to process these queries quickly.

6.2 Large Query Queue (Q2)

This queue is for any query submitted through the web interface that is larger than a “standard” query. Typically, these queries will be region searches for regions larger than the “standard” query or all-sky queries with energy cuts. These queries typically take longer and have a less stringent requirement on completion time.

This queue has medium priority and there will always be at least one processor available to this queue for processing queries.

6.3 Batch Query Queue (QB)

This final queue is for all queries submitted in a **TBD** batch mode or through the command line interface. Any query may fall into this category. This queue has the lowest priority and will only run when there are available processors when the other queues are empty.

6.4 Queue Scheduling Algorithm

The goal of the scheduling algorithm is to complete the most number of queries in the least amount of time and minimize the time any one user must wait for their query to complete. To this end, it attempts to utilize as many of the connected search nodes as possible in an efficient manner. It is not restricted to utilizing only a single search node per query but may divide the data search for a query among multiple nodes if they are available.

The scheduling algorithm for selecting queries and scheduling them on the compute nodes is defined by the following rules:

1. No single query can use all the search nodes for processing.
2. At least one (1) node must be reserved for Q2 queries if there are any queries in that queue.
3. Attempt to clear Q1 queries as quickly as possible.
4. Queries in QB are only processed when there are available search nodes after starting all Q1 and Q2 queries.
5. All internal HTM pixel data files for a single time interval must go to the same search node.

6. All time intervals sent to a single search node must be contiguous.
7. Do not split the query among multiple search nodes if the total number of photons to search times the data line size in bytes is less than 1.25 times the size of the MAX_FILE_SIZE parameter value or 250 Mbytes (**TBR**), whichever is less.
8. Do not split query among multiple search nodes if the estimate completion time is less than 20 (**TBR**) seconds.
9. Subject to the above rules process the queries on as many search nodes as possible, dividing the time intervals as evenly as possible among the nodes in order to minimize the time the user must wait for the results.
10. Queries split between multiple search nodes must be split on time interval boundaries with the node getting the first chunk given ID=0 in the NEW_SEARCH message, the node getting the second interval getting ID=1 and so forth.

A typical standard search for one year's worth of data on the photon database could return anywhere between 50 and 600 Mbytes of data. Thus the query would typically be split between 1-3 search nodes. On a 10 node cluster, between 4-10 queries could be processed simultaneously with no loss of processing speed.

7 Detailed Design

This section expands on the information presented in section 2 and, using information presented in the previous sections, provides a detailed description of how the various components of the database system work and interact with the user and one another.

7.1 User Interface

The user interface will consist of two parts. The first is the web interface that the general scientific community will use to access the data. The second is a command line interface that will allow the system administrators to send commands to the system and query the system status as well as submit data search queries identical to those submitted through the web interface.

7.1.1 Web Interface

This is the main user interface to the data system and is modeled on the HEASARC Browse database interface. The web interface consists of two scripts, one for entering the query parameters and one for viewing the results of the query. The scripts are written in Perl CGI and hosted on the FSSC website. There will also be a web page providing detailed instructions for entering a query.

7.1.1.1 Program Flow

The program flow for the web interface is shown in Figure 5. First the LATDataQuery.cgi script is called which presents a web page that allows the user to input the query parameters (see section 7.1.1.4 below). Once the parameters have been entered, the script validates the entries. If the entries are invalid, the page redisplay with an error message allowing the user to correct the error. If the parameters are valid, the script will generate a query ID string for the query (see section 7.1.1.2 below), compose a NEW_WEB_QUERY message and send it to the QueueManager with the query parameters. The content of this message is described in section 7.1.1.3 below.

Once the message has been sent to the QueueManager, the LATDataQuery.cgi script waits for several messages in response. The first is a message indicating either that the message was properly received and validated by the QueueManager or that it could not be validated. If the query is accepted by the QueueManager, the script will receive a WEB_QUERY_RECIEVED message. Otherwise, a BAD_QUERY message will be received followed by a DISCONNECT message closing the connection. If a BAD_QUERY message is received, the script will redisplay with a message indicating the problem, and allowing the user to modify the entries and resubmit the query. The typical cause of the BAD_QUERY message will be because of a server being offline or a data ingest occurring.

After receipt of the WEB_QUERY_RECEIVED message the script will wait for another message indicating either that the query has been entered into a query queue or that it is completed. Typically the former will be the case but for time range only searches, the query will complete quickly enough that the latter message will be received. If the query is queued, the QueueManager will send a QUERY_RECIEVED message to the script indicating that the query is in the queue and giving an estimated time until completion. If the query has finished, the QueueManager will send a QUERY_FINISHED message. The receipt of either of these messages will be followed by a DISCONNECT message and will cause the script to close its

connection to the QueueManager and display the Query Submission Status web page described in section 7.1.1.4.2 below. This page will include a link to the QueryResults.cgi script described in section 7.1.1.5 below. The link will indicate the query ID of the query in question to allow the retrieval of the query results without additional input from the user.

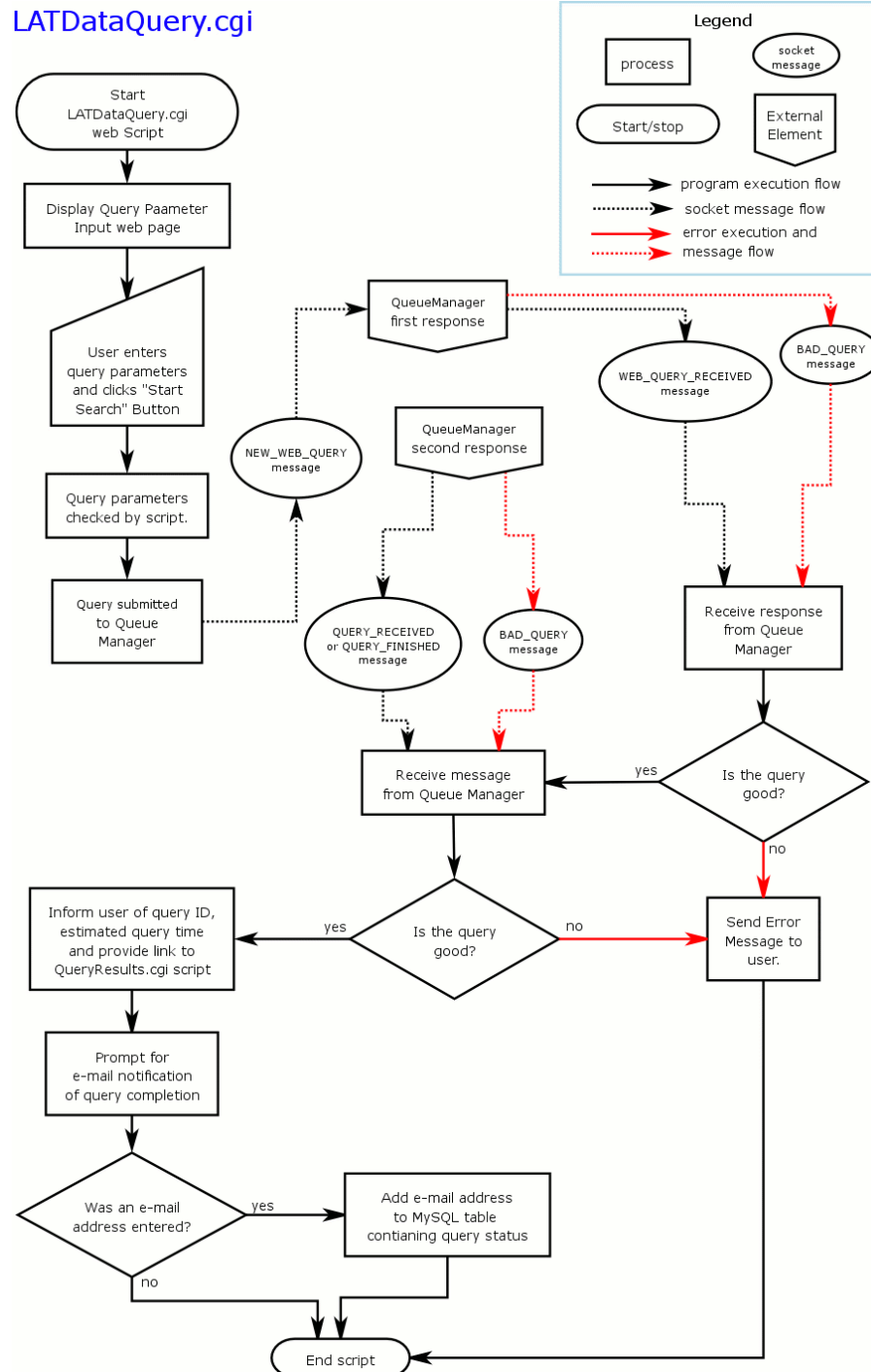


Figure 5 - Program flow for LATDataQuery.cgi script

Finally, the Query Submission Status web page will provide a small form to allow the user, if desired, to input an e-mail address at which to be notified when the query is complete. This is primarily intended for use with queries to the event database which may take several hours to complete, but can be used for any search.

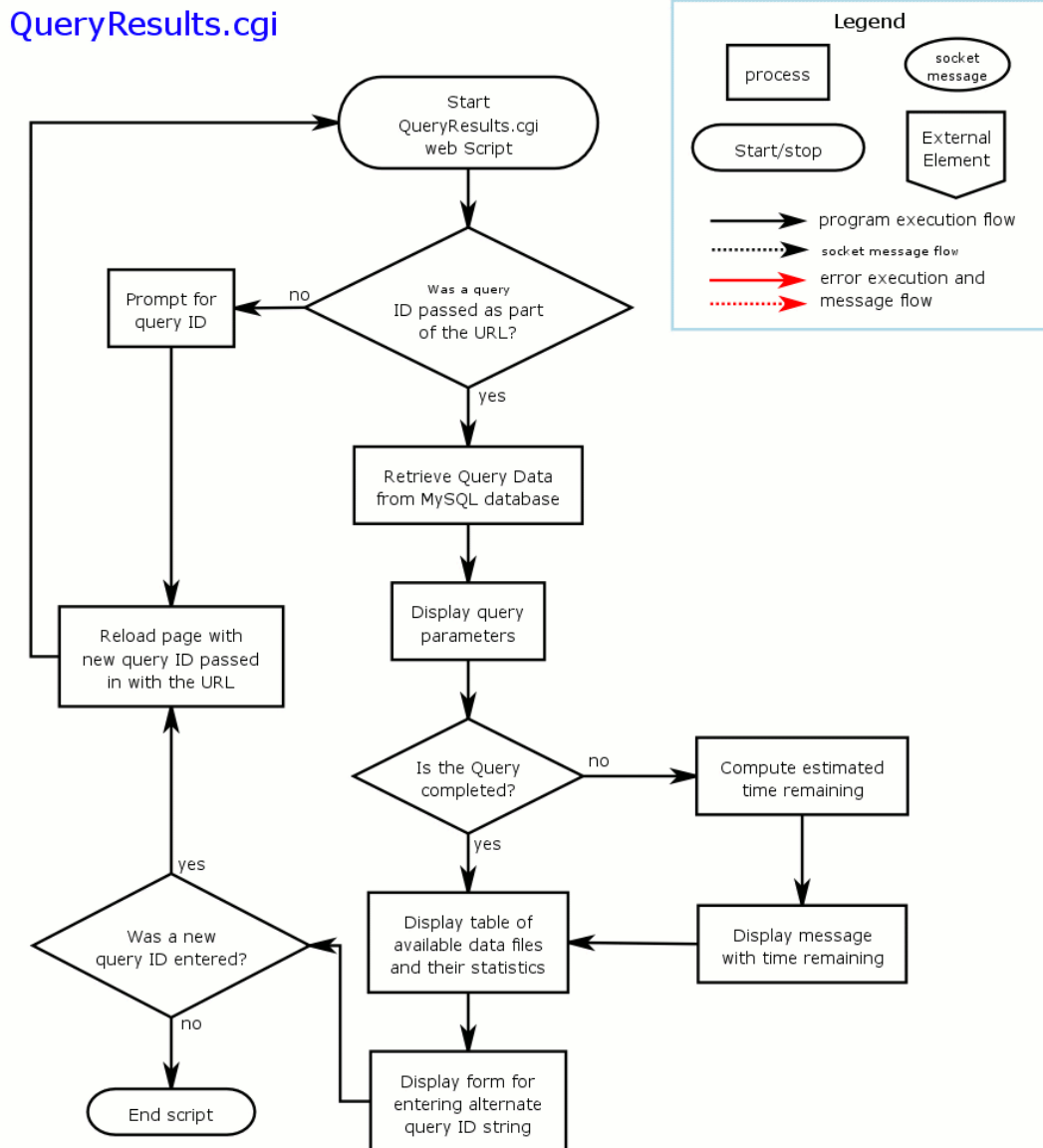


Figure 6 - Program flow for QueryResults.cgi script

Figure 6 shows the program flow for the QueryResults.cgi script. Once the user clicks on the link to the QueryResults.cgi script, this script will query the MySQL database for the status of the query. If the query is not yet finished, the user will be informed of the estimated time remaining until the query is finished and presented with a list of any files that are available so far and a form to enter an e-mail address for notification when the query is finished, if one has not already been entered. If the query is completely finished, the user will simply be presented with

a list of all the data files generated by the query with various file statistics. In either case a summary of the query parameters will be presented as well. Each entry in the file list will be a link to the Fermi FTP site which the user can click on to download the file. Finally, a small form will be presented at the bottom of the page to allow the user to enter the query ID of another query to get the results for.

If accessed directly (not via the LATDataQuery.cgi) script, this script will provide a page that only contains the form to ask the user for the ID of the query they want to see the results of. Once this has been entered, the script behaves exactly as described in the proceeding paragraph.

7.1.1.2 Query ID String Format

Each query will be assigned a query ID to distinguish and identify it in the database system and log tables. The format used should generate an ID that is unique for the life of the Fermi mission and beyond. This ID is used throughout the system to identify messages pertaining to the given query, to find the query results and parameters in the MySQL data tables and forms the basis of the filenames of the output data for most queries. The format used is as follows:

LYYYMMDDHHmmSSXXXXXXXXNN

where the various fields are:

- L – indicates it is a query for the LAT instrument
- YYYMMDD – the date the query was submitted, 4 digit year, two digit month and two digit day.
- HHmmSS – the time the query was submitted in hours, minutes and seconds, two digits for each element
- XXXXXXXX – a hexadecimal number based on the IP address of the host submitting the query, manipulated and scrambled for privacy but in a way that preserves uniqueness.
- NN – a two digit random number just in case we get multiple requests from the same IP address in the same second (possible with proxies and such).

7.1.1.3 Query Submission Message Format

When the query parameters entered by the user have been verified, the LATDataQuery.cgi script will send off a NEW_WEB_QUERY message to the QueueManager. The ID string field in the message header will contain the Query ID string generated for the query. The message size will be 2 and the body will contain 2 entries. The first entry will be a string containing the query parameters as described below. The second line will contain an identifier indicating the source of the query. In the case of the LATDataQuery.cgi script, this identifier will be “DC2Interface”. The QueueManager will use this information to assign a priority to the query.

The format of the query parameter list is a series of “keyword=value” pair separated by double ampersands (‘&&’) and terminated with the bare ‘END’ keyword. Table 5 lists the allowed keywords, their possible values and a short description of the parameter. The keyword names are case sensitive.

Table 5 - Keywords and values for the query parameter string to be sent to the QueueManager

Keyword	Possible Values	Description
database	'event', 'photon' or 'spacecraft'	This keyword describes the databases that the query should be sent to. It may be used multiple times within a query to indicate each of the database servers that should receive the query parameters. The event and photon database can no longer be queried simultaneously. If both of those databases are sent to the QueueManager, it should result in an error.
shape	'circle', 'box' or 'all-sky'	This indicates the shape of the search region. If circle is specified, the RA, Dec and radius parameters must be specified as well. If box is specified, the RA, Dec, length, height and rotation parameter must be specified as well. If all-sky is specified, there will be no additional position parameters. Currently only a circle search is supported.
radius	(0, 90]	The radius of the search region in degrees if a circle is selected.
length	(0, 360]	The length (width) of the search region in degrees if a box is selected.
height	(0, 180]	The height of the search region in degrees if a box is selected
rotation	[0, 360)	The rotation angle of the box in degrees. The angle is measured such that a positive value represents a counterclockwise rotation.
RA	[0, 360)	The Right Ascension of the center of the search region
Dec	[-90,90]	The Declination of the center of the search region
minMET	[0, ∞]	The start time for the search in MET seconds. Assumed to have a value of 0 unless specified. If specified, must have a value less than the maxMET parameter.
maxMET	[0, ∞]	The end time of the search in MET seconds. Assumed to have a value of ∞ unless specified. If specified, this must be larger than the value of the minMET parameter.
energyMin	[0, ∞]	The lower energy limit of the search in MeV. Assumed to have a value of 0 unless specified. If specified, must have a value less than the energyMax parameter.
energyMax	[0, ∞]	The upper energy limit of the search in MeV. Assumed to have a value of ∞ unless specified. If specified, this must be larger than the value of the energyMin parameter.
END	--	This keyword has no value and terminates the query string.

Only those parameters needed to specify the query are included in the query parameter string. All others will be ignored or assumed to be their default values by the database servers if not specified. As an example, assume the user wished to perform a 10° radius search centered on the Crab pulsar (RA=83.6, Dec=22) for all photons with energies greater than 1 GeV and get the spacecraft data as well. The query string would look like

```
database=photon&&database=spacecraft&&shape=circle&&RA=83.6&&Dec=22&&radius=10&&energyMin=1000&&END
```

This would retrieve all the photons from the last six months of the mission that fell within this region and had an energy greater than 1000 MeV (= 1 GeV) and retrieve the spacecraft history from the same time frame.

7.1.1.4 LATDataQuery.cgi Script

The LATDataQuery.cgi script is the first script encountered by the user when they wish to extract data from the LAT event, photon and spacecraft databases. This script presents two web pages to the user, the query parameter input page and the query submission results page.

7.1.1.4.1 Query Parameter Input page

The initial page presented by the script will provide the user with the following information and options, which are described in the subsections below:

- A link to the help page for entering queries
- A short summary of the date ranges covered by the data in the database and the total number of photons and events in the database.
- Fields for entering query parameters. Currently, search may be performed on sky position, time and energy. Each field will provide a link to the relevant section of the help page that provides a short summary of the type/format of data to enter into the field.
- Choice of which database(s) to search.
- Buttons to submit query parameters or clear the form.

7.1.1.4.1.1 Link to help

At the top of the initial query page will be a link to the help web page describing the details of how to enter data queries. This link will take you to the top of the help page. Additionally, the field names for the various query parameters will be links to the individual sections of the help page that describe the options for the query parameter in question.

7.1.1.4.1.2 Data summary

Also near the top of the page, the script will provide the user with a summary of the data contained in the databases including the dates of the first and last data entry as well as the number of photons and total events contained in the database.

7.1.1.4.1.3 Query parameter fields

The database allows users to query for data based on sky position, time and energy range. For each of these parameters there will be a field on the page for the users to enter values.

7.1.1.4.1.3.1 Sky Position

Here the user may enter the coordinates of the center of the sky region to search as well as the size and shape of the region. If no sky position is given, data from the entire sky will be returned.

For entering the center of the sky region, the user will have the choice of J2000, B1950, or galactic coordinates. Values will be allowed in standard astronomical formats. Both coordinates will be entered in the same field with a comma separating the two values. The type of coordinate entered will be selected by a pull-down box next to the field for the coordinate entry. Alternately, the user may simply type in the name of an object. This will cause the script to look up the name from the Simbad server and submit the coordinates returned.

For entering the size and shape of the search region, the user may choose between circular and box searches. The form will provide a field for entering the dimensions of the region with a pull down box for the shape. If a circular region is selected, the user will enter the radius of the search area in the area field. If a box is chosen, the user will enter the length, width and rotation angle for the box separated by commas.

The default values, which will be presented when the page loads, are to use the J2000 coordinate system with a circular search area, 15° in radius.

7.1.1.4.1.3.2 Time Range

Here the user may specify the time range to cover in the search. If no time range is specified, the last 6 months of data will be returned.

A single field will be used to enter the start and stop time for the search. The stop time is optional and if not supplied, the system will assume a stop time equal to the start time plus 6 months or the end of the data, whichever is less. If the stop time is supplied, it should be separated from the start time by a comma.

Time may be entered in one of three formats, Mission Elapsed Time (MET), Modified Julian Date (MJD) or standard Gregorian dates. MET is entered in seconds. MJD is entered in fractional days and Gregorian time is entered in DD-MM-YYYY HH:MM:SS format. The default will be Gregorian time.

Two special entries will be provided, START and END. These are case insensitive and refer to the start and end of the data currently in the archive. Thus to search for data from the entire mission the user could simply enter “start, end” in the time field.

7.1.1.4.1.3.3 Energy Range

Here the user may specify the energy range to limit the search to. If no energy range is entered, a default energy cut will be done based on values in the configuration file for the front end. This means that all queries will search by energy.

A single field will be used to enter the lower and (optional) upper energy limits. If the upper energy limit is entered, it must be separated from the lower energy limit by a comma. If only a lower energy limit is entered, all events with energies greater than that energy will be returned. To search for events below a specified energy a value of zero should be entered for the lower limit. All energies are entered in MeV.

7.1.1.4.1.4 Database choice

This portion of the page allows the user to select which of the databases they would like to submit the query to. The photon and event database are selected via a drop down box allowing only one or neither of the two databases to be selected. The spacecraft database is selected by a checkbox that is checked by default. The query will be submitted to the selected databases. By default the photon and spacecraft databases will be selected

7.1.1.4.1.5 Form Buttons

There will be two buttons, one to submit the search and the other to reset the page and clear all the fields.

The first button, labeled “Start Search” will cause the query parameters to be validated. If all the parameters are valid, the query will be packaged in to a NEW_WEB_QUERY message (see section 5.3.1) and sent to the QueueManager and the Query Submission Status page will be

displayed. The details of the body of this message are described in section 7.1.1.3 above. If there are errors, a message will be displayed describing the error and the Query Parameter Input page will be redisplayed.

The second button, labeled “Reset Form” will cause all the fields, pull-down boxes and checkboxes to be reset to their default state.

These buttons will be located both at the top of the page above the sky position input area and at the bottom of the page below the database selection area.

7.1.1.4.2 Query Submission Status page

Upon receipt of the QUERY_RECEIVED or QUERY_FINISHED message and the following DISCONNECT message, the LATDataQuery.cgi script will display the Query Submission Status page. This page will provide a summary of the query parameters submitted to the database system, the query ID string generated for the query, and a link to the QueryResults.cgi script that includes the query ID string so that the query results will be posted by clicking on the link.

If the QUERY_RECEIVED message was received, the script will extract the queue position and estimated time to query completion and present that information to the user before the link to the results page. Additionally, the page will provide a form to allow the user to input an e-mail address at which to be notified when the query is done.

If the QUERY_FINISHED message was received, the page will inform the user that the query is complete and directing them to the results page via the provided link.

7.1.1.5 QueryResults.cgi Script

This script presents the user with a web page listing the current status and results of the query. It can be called in two ways. The first, and less typical way, is to call the script without any additional parameters appended to the URL. This will cause the script to display a page containing a form that prompts the user to enter the ID string for the query they wish the results of. Once the string is entered, the script is reloaded with the query ID string appended as a query parameter to the URL.

This is the second method of calling the script and will be the one most commonly used as it is the form used by the LATDataQuery.cgi script to link to this page. In this case the query ID string is loaded and the results are extracted from the MySQL database. If the query is not found in the MySQL database, the user is notified and the form prompting for a query ID string is displayed to allow them to enter a different string. If the query is found all the relevant information about the query is extracted from the database and presented to the user.

First, the user is given a summary of the exact query parameters used to extract the data as these may be slightly different (in the case of all-sky searches) than what the user input (see section 7.3.1.4.2.1). Next, if the query is not completely finished, the user will be informed of the estimated amount of time remaining before completion, otherwise, a statement will be presented indicating that the query is complete. After this, a table will be presented listing the files containing the requested data that are currently available. The files will be grouped by database (i.e. event, photon, and/or spacecraft) and will list the filename, file size, md5sum checksum and number of data entries for each file. There will be a link to the file on the GSSC’s website. At the bottom of the page will be the form to enter the query ID string of a different query.

7.1.2 Command Line Interface

This interface is for those maintaining the database system and for testing. It allows the user to submit queries just like the web interface but also allows the user to query the status of the various servers, get statistics about the server and usage and stop or suspend various parts of the server system.

More info TBD.

7.2 QueueManager

The QueueManager is the interface between the database servers and the user interface software and controls the distribution of query requests to the various database servers. It performs validation on the query parameters and assigns the queries to the various priority queues described in section 6 above. The QueueManager also monitors the connections to the various database servers and attempts to reconnect if the connection is lost.

The QueueManager is implemented as a single compiled C++ program that communicates with the other elements via TCP/IP sockets and maintains metadata in a MySQL database. The following subsections describe the various aspects of the QueueManager's operation.

7.2.1 General Program Flow

Figure 7 presents the basic flow of the QueueManager program. At startup, the QueueManager begins by reading its configuration file to determine the location of the MySQL database, the LAT database servers and the port it should listen on for connections from the user interface tools. After reading this information, the QueueManager establishes a connection to the MySQL database and reads the data on outstanding queries and creates its initial list of open queries.

After the data has been read from the MySQL database, the QueueManager establishes a connection with each of the three database servers and adds the connections created to its list of open TCP/IP sockets. It then opens the listen port for incoming connections from the user interface tools and adds this port to its list of open sockets. At this point the QueueManager is ready to accept queries and messages from the servers and enters a loop that it will remain in until it receives a PROGRAM_CONTROL:SHUTDOWN message from the command line tool.

This loop begins with the QueueManager verifying the connection to each of the three database servers. If any of the servers are disconnected, an attempt is made to reconnect. If the attempt fails a timer is set with the length equal to the time specified in the CHECK_INTERVAL parameter in the configuration file. If no CHECK_INTERVAL parameter is specified, the QueueManager will use the default value of two minutes (120 s). If all the connections are good, the timer is set to infinity. The QueueManager then goes into a blocking mode to wait until either the timer expires or one of the connected sockets cause a trigger.

If the timer expires before a socket is triggered, the QueueManager will simply reenter the loop from the beginning and attempt to reconnect to the servers that have lost connectivity. This will continue until all the servers are reconnected.

If one of the connected sockets trigger, the QueueManager will react appropriately depending on which one triggered. The connected sockets will trigger for one of three reasons. First, one of the user interface tools will try to connect to the listen port, causing this port to trigger. Second, a message will arrive from one of the connected servers or user interface tools.

Finally, if one of the connected sockets closes unexpectedly, this will cause the socket to trigger as well.

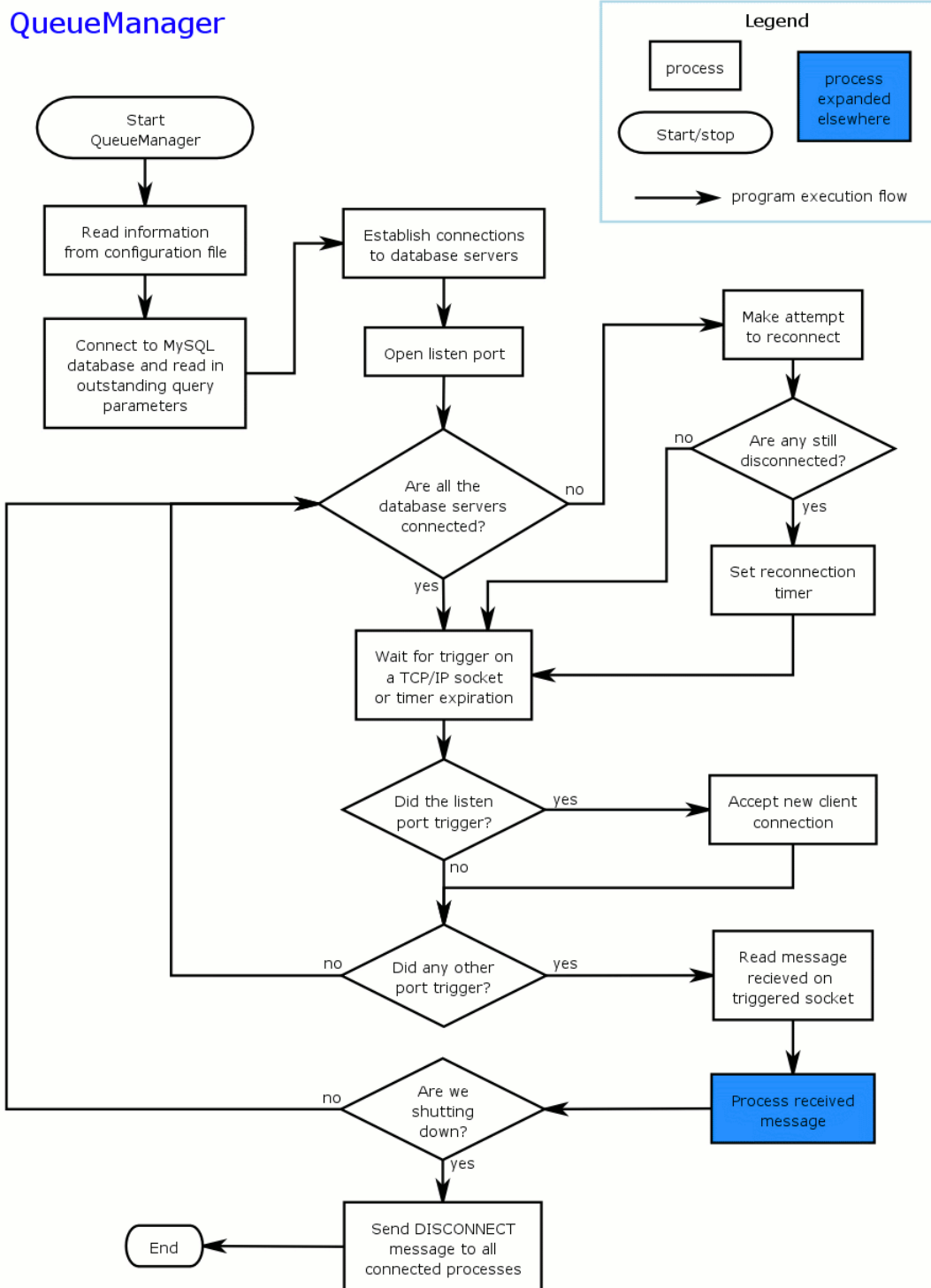


Figure 7 - Basic program flow for the QueueManager

If it was the listen port that triggered, the QueueManager will accept the new connection from the user interface and add the new connection to the list of open sockets. If a new message from one of the established socket connections causes the trigger, the arriving message will be read and processed appropriately. The details of the effects of each received message are presented in section 7.2.2 below. Finally, if one of the sockets unexpectedly closes, the QueueManager will treat this just as if it had received a DISCONNECT message and remove that connection from its list of open connections.

7.2.2 Message Processing

This section describes the actions that should be taken by the QueueManager for each of the various messages that it receives.

7.2.2.1 NEW_WEB_QUERY

This should be the first query received after a new connection is established on the listen port. When the QueueManager receives this message it will first verify the query parameters and check to see that all the servers needed to perform the query are connected. If there is a problem with the query parameters, a BAD_QUERY message will be composed and sent back to the user interface informing the user of the problem with the parameters. If one or more of the necessary servers are down, a BAD_QUERY message will be composed and sent to the user interface tool informing the user that the servers are currently down and that they should try again later.

If all is well, the QueueManager first sends a WEB_QUERY_RECEIVED message back to the user interface. It then creates an entry in its list of open queries for the new query. The QueueManager then determines which priority queue the query should be in and creates a NEW_QUERY message that contains the query information and the priority queue, creates an entry for the query in the MySQL database, and sends the NEW_QUERY message to each of the servers involved in the query.

7.2.2.2 QUERY_RECEIVED

When this message is received, the QueueManager know that the server that sent the message successfully received the query and added it to the proper priority queue. The body of this message contains the position number in the queue and the estimated time to completion for this server and this query. Once all the QUERY_RECEIVED messages from all servers have been received by the QueueManager, the contents of all the messages are collected into a single QUERY_RECEIVED message that is then sent back to the user interface to inform the user of when the query should be completed.

In the case of all-sky searches, this message is not sent to the user interface tools. Instead a QUERY_FINISHED message will be sent instead (see section 7.2.2.4 below).

After this message is sent to the user interface tool, it is followed immediately by a DISCONNECT message informing the user interface that the QueueManager is closing the connection. The QueueManager then closes the socket and removes the socket from the list of open connections.

7.2.2.3 QUERY_STARTED

This message is sent from the database server when the query has reached the top of its priority queue and processing has started on the query in question. When this is received, the queue manager updates the state of the query in its internal data tables.

7.2.2.4 QUERY_FINISHED

This message is received when a server has completed processing the query in question. When this message has been received from all servers involved in the query, the QueueManager checks to see if an e-mail notification has been requested. If so, an e-mail is sent informing the user that the query is complete. The query is then removed from the QueueManager's list of open queries.

Typically, this message is not forwarded back to the user as the connection to the user interface tool has closed by the time the QueueManager receives this message. However, in the case of all-sky searches, this message will be received before the connection is closed. In this case, once the QueueManager has received the QUERY_FINISHED message from all the servers involved in the all-sky search, a QUERY_FINISHED message is sent back to the user interface.

After the QUERY_FINISHED message is sent to the user interface tool, it is followed immediately by a DISCONNECT message informing the user interface that the QueueManager is closing the connection. The QueueManager then closes the socket and removes the socket from the list of open connections.

7.2.2.5 BAD_QUERY

This message is received from a database server if there is a problem processing the query requested. The body of the message contains a description of the problem. When this message is received, the QueueManager also removes the query from its list of open queries and this message is forwarded to the user interface.

After this message is sent to the user interface tool, it is followed immediately by a DISCONNECT message informing the user interface that the QueueManager is closing the connection. The QueueManager then closes the socket and removes the socket from the list of open connections.

7.2.2.6 DISCONNECT

This message is received when one of the connecting sockets is going to close down. Receipt of this message causes the QueueManager to close its end of the socket and remove the socket from the list of open connections.

If one of the sockets triggers due to an unexpected closure on the other end, it is treated as if the QueueManager had received the DISCONNECT message and the socket is closed and removed from the open socket list.

7.2.2.7 PROGRAM_CONTROL

The effects of this message depend on the content of the message body. The effects of the various commands that are received in the PROGRAM_CONTROL message are described in the following subsections.

7.2.2.7.1 Shutdown

The receipt of this command is a signal to shut down the entire system. When the SHUTDOWN command is received, the QueueManager starts by closing its listen port so that no new connections can be established. It then forwards the message to all the connected database servers. Once this message has been sent, the QueueManager then waits to receive

DISCONNECT messages from each of the database servers. While it is waiting it will continue to process any other messages it receives from the servers relating to queries that are finishing up. As the DISCONNECT messages are received from each server, the connection to that server is closed. Once all the server connections are closed, the QueueManager sends a DISCONNECT message to any remaining client connections and closes those connections. It then stores any state information needed (TBD) and shuts down.

7.2.2.7.2 Status

(This message has never been implemented.) The receipt of this message is a request to know the current state of the data system. When the STATUS command is received, the QueueManager will forward the message to each of the database servers as well. The QueueManager will then compose a STATUS_RESPONSE message to send back to the requesting user tool. The body of this message will contain the information described in Table 6.

Table 6 - Contents of the QueueManager portion of the STATUS_RESPONSE message

Body line number	Contents
1	Text string 'QueueManager'
2	Date & time stamp of server start up
3	Total time alive in HH:MM::SS format
4	Number of queries completed in '%d %d %d %d' format where the first number is the total and the other three are the counts for the individual priority queues.
5	Average time (in seconds) per query in "%f %f %f" format, one per queue
6	Number of currently open queries in '%d %d %d %d' format where the first number is the total and the other three are the counts for the individual priority queues.

The QueueManager will then wait for the STATUS_RESPONSE messages from each of the database servers. The body of these messages (see section 7.3.1.4.8.2 below) will be added to the STATUS_RESPONSE message containing the QueueManager's status in the following order:

1. Event database status
2. Photon database status
3. Spacecraft database status

If any of the above servers are not connected at the time the STATUS message is received, the QueueManager will add two lines to the body of the message in place of the status report. The first line will be the name of the server (i.e. 'Event', 'Photon' or 'Spacecraft') and the second will be the text string 'Not Connected'.

Once the STATUS_RESPONSE messages from all the servers have been received and the final response message has been composed, it is sent back to the user tool that requested the status report.

7.3 Database Server

The database servers for each of the databases are based on the same core software design regardless of the type of data that the database handles. In this section we describe the

design of this system as it applies to all three servers and note the unique differences where they occur.

Each server consists of two main components, a control node and one or more search nodes. The system is designed to run on a Linux Beowulf cluster with the control node process running on the cluster's head node and the search node processes running on the compute nodes of the cluster. The following subsections describe the operation of these components.

7.3.1 Control Node

The control node is the interface between the QueueManager and the search nodes and is primarily responsible for scheduling the execution of the queries in the query queues. It maintains the query queues and implements the queue priority policies described in section 6 above. The control node monitors the status of the search nodes connected to the system and is responsible for distributing the query processing among these nodes.

The control node maintains an internal metadata table with information on all the internal data files use by the database system. It uses this information to determine the files needed to complete a query and to make an estimate of the time needed to complete the query.

The control node is implemented as a single compiled C++ program that communicates with the other elements via TCP/IP sockets and maintains metadata in a MySQL database. The following subsections describe the various aspects of the control node's operation.

7.3.1.1 General Program Flow

The basic program flow of the control node is nearly identical to that of the QueueManager and is depicted in **Error! Reference source not found..** The control node process starts by reading the configuration information from its configuration file. The data in the configuration file inform the process of what port to listen on for connections, where the data is located, the location of the MySQL database tables and the type of data the node will be processing (event, photon, or spacecraft).

After the configuration information has been loaded, the control node will then connect to the MySQL database and read in the metadata for the internal data files for that server. This information is used to populate an internal metadata table that will be used during query processing. Once the file metadata has been loaded, the control node then queries the database for the query metadata of all the uncompleted queries. These queries are loaded into their appropriate query queues and processing of the queries will start as soon as there are search nodes connected to the system.

At this point the control node process enters a loop where it waits for connections from the search nodes and QueueManager to be established, messages to be received on established connections, and starts queries that are pending. First the control node waits for one of its open sockets to show activity or the wait timer to expire. If it is the listen port that triggered, the new connection will be established and the connection added to its list of open connections. For all other connections that trigger, the received message will be read and processed. Once all the new messages have been read, the control node then checks to see if there are any queries to process.

Figure 8- Basic program flow for the control node process of the database system

If the queues are empty of pending queries or all the search nodes are busy, the program just goes back to waiting for the sockets to trigger. If there are available search nodes and the queues are not empty, the control node will determine which query to process next and launch that query on the appropriate number of search nodes. If there are more pending queries and open search nodes the control node process sets a short (less than 1 second) timer and goes back to waiting for the sockets to trigger. If there are pending queries, then the timer will likely expire before another message arrives at which point the control node will simply start the next query. If the search nodes are all busy or the queues contain not pending queries, no timer will be set and the control node will simply wait for a trigger on one of the sockets. The process of starting a new query is described in greater detail in section 7.3.1.3 below.

If a PROGRAM_CONTROL:SHUTDOWN message is received, the control node will shut down as soon as all currently processing queries are completed.

7.3.1.2 Internal Metadata Tables

The control node maintains three important internal metadata tables. The first contains information about the time bins the data has been partitioned into. The second contains information on all the internal data files. The final table contains status information on all the search nodes connected to the database system. The details of these tables are described in the following subsections.

7.3.1.2.1 Time Metadata

This table holds information about the various time ranges the data has been partitioned into. Each entry contains the following information:

- A string that is used as the beginning of the internal data file filename.
- The start and stop time of the file.
- A string containing the version number of the files in that time interval. This value is appended to the end of the filename after the HTM pixel name and before the '.fits' extension.

7.3.1.2.2 File Metadata

This table holds information about the size and contents of the internal data files. It is indexed by the filename and contains, for each entry, the following information:

- Number of events contained in the data file.
- The start and stop time of the data in the file.

7.3.1.2.3 Search node status metadata

This holds information about the search nodes that are connected to the system. It is indexed by the socket number the search node is connected to and holds the following information:

- Node identifier
- Date connected
- Flag indicating whether or not the node's copy of the internal data files is up to date.
- Status sting containing the state of the system. It will contain the query ID of the query the node is currently processing, 'idle' if no query is being processed or 'ingest' if the node is currently ingesting new data.

- Node number for the query being processed.
- Number of queries processed
- Total time spent working on queries

7.3.1.3 Starting a New Query

After the new messages have been processed, the control node must decide whether or not to start processing a new query on the search nodes. When it reaches this point in the execution there will be one of three conditions:

1. No pending queries to process. Either there are no queries at all or they are all currently being worked on.
2. Pending queries but no available search nodes. In this case, there are queries to be processed but the search nodes are all busy working.
3. Pending queries and idle search nodes. In this final situation a new query will be started on the search nodes.

In the first case, there is nothing for the control node to do so it will just go back to waiting for another event on one of its socket connections. In the second case, there are no available resources to process queries. Again there is really nothing the control node can do until more resources become available. This will be signaled by a SEARCH_FINISHED message from one of the search nodes so again the control node just goes back to waiting for a socket to be triggered.

Only in the final case will the control node actually take action. The first thing the control node does is to determine which query is to be processed next. This is done by implementing the priority rules described in section 6.4. Once the query to process has been determined, the control node will decide the number of search nodes to use to process the query. This will be a function of the number of available nodes, the size of the query and the priority queue the query is in.

Using this information, the control node divides the list of internal data files to search among the search nodes that will process the query. For each node a NEW_SEARCH message is generated that contains the FITS row filter to apply to the data and the list of files to search. This message is then sent to the appropriate search nodes that will then process the query. At this point the control node is done with this query until the results return.

Once all the NEW_SEARCH messages have been sent out, the control node checks to see if there are any other queries still in the queues. If there are and there are available search nodes to process them, the select() command, used to check for new socket messages, is set to be non-blocking. It will check to see if new messages have arrived while the last query was started but otherwise drop right back to this portion of the program flow if none have arrived and start processing another query.

7.3.1.4 Message Processing

The control node handles a wide variety of messages from many different sources. This section describes the messages received and the actions the control node takes when it receives them.

7.3.1.4.1 IDENTIFY

If this message is received from the QueueManager, a flag is set indicating that the QueueManager is connected and all messages that should go to the QueueManager will be transmitted. If it is received from the ingest program, no special action is taken.

If received from a search node, an entry for the search node will be made into the search node metadata table and the name included in the body of the message will be stored in the table. This node is then marked as available for processing queries.

7.3.1.4.2 NEW_QUERY

This is the message that the control node receives from the QueueManager to signal the arrival of a new query. When this message arrives, the action of the control node depends on the type of query. Queries can fall into one of three broad categories:

1. Time only searches – These queries want data from the entire sky from a particular time range. These queries are processed quickly using only metadata and don't actually end up in a query queue. (*This functionality should be checked.*)
2. All-sky searches with energy limits – These are like the queries above but also have energy cut parameters specified. In this case the all-sky data will be used to do the search. Typically, these queries end up in the large query queue unless they cover a very small time period. (*This functionality should be checked.*)
3. Region searches – These are queries that have specified a particular region of the sky from which to extract data. These queries make use of the HTM pixilated data to speed up their search and could end in any of the query queues depending on the details of the search parameters.

The details of the control node's behavior for each of these queries are described in the sections below.

7.3.1.4.2.1 Time range only queries

If the query contains only search parameters dealing with time, the query can be completed using only metadata and not actually searching through the data files in detail. These queries return quickly and do not enter into the priority queues although the QueueManager assigns them the highest priority queue when sending the NEW_QUERY message.

For these queries, the control node simply looks at the time metadata for the all-sky data and extracts the list of files that cover the time range specified. Using this list of files, a QUERY_FINISHED message is generated and sent back to the QueueManager and the query is marked as completed.

These queries actually return more data to the user than requested as no trimming of the data set is made on the first and last files in the list which will contain data before and after the specified time range respectively.

7.3.1.4.2.2 All other queries

If the query includes search parameters other than time, the control node does several things:

1. Adds the query to the appropriate queue
2. Generates the FITS row filter
3. Generates list of files to search
4. Estimates the time needed to complete the query

The first thing the control node does when it receives the NEW_QUERY message from the QueueManager is to add the query to the appropriate query queue. The queue that the query should be placed in is specified in the third line of the NEW_QUERY message body.

Once the query is in the queue, the control node parses the query string provided on the first line of the NEW_QUERY message body and generates the appropriate FITS row filter that will extract the appropriate data for the query. This row filter is stored as a string and uses the row filtering syntax specified in the FITSIO user's manual.

Next, the control node determines the internal data files that will be needed to perform the query. If the query is an all-sky query with energy cuts the file list is extracted from the all-sky files metadata using the time range information in the query. For these searches the file list is generated by finding all of the all-sky data files that cover the time period in question and marking them as partially enclosed by the search boundaries.

If the query includes a spatial region, some more work needs to be done to create the file list. For region searches the file list is created by using methods provided by the HTM interface and the server's metadata. From the search area defined by the query, an HTM domain is constructed. Using this domain and the HTM_LEVEL specified in the configuration file, an HTM function is called that returns two lists. The first list contains all those HTM pixels that are fully enclosed within the search area. The second list contains all those HTM pixels that are partially enclosed within the search area, i.e. the boundary of the search region passes through the pixel.

Next, using the time boundaries of the search the relevant time bins needed to process the query are extracted from the time metadata. This information, combined with the HTM node information is used to construct the list of files needed to perform the query. In the list, each file is marked as to whether the data in the file is fully or partially enclosed by the search boundaries. This is determined in the following manner.

First all files with data from pixels in the partially enclosed list are marked as partially enclosed. Next if there are search parameters dealing with time in the query (i.e. the time field on the web interface is not START,END) all files from the first and last time bin are also marked as partially enclosed as some events in those files may lie outside the search boundaries. Finally, if there were any energy cuts applied, all files are marked as partially enclosed as the energy cuts will need to be applied to all events in all the files to get the query results.

Once the full list of files to search is generated, the number of events stored in each file is extracted from the file metadata. These values are summed to get the total number of events that need to be searched and this value is used to estimate how long the query will take to process.

At this point the control node generates a QUERY_RECEIVED message that contains the position in the queue (*is the position in the queue value actually implemented?*) and the estimated time to complete the query and sends it back to the QueueManager.

7.3.1.4.3 SEARCH_FINISHED

This message is received from a search node when it is done processing its portion of the query. When this message is received, the control node does two things. First, it updates the search node metadata for the node it received the message from to indicate that the node is free to be used on another query. Second, the control node extracts the output data file name from the message body and stores it in the list of output files for the query the message corresponds to.

The control node then checks to see if there are any search nodes still working on this particular query. If there are, no more processing is done. If this was the last (or only) node working on the query the control node generates a QUERY_FINISHED message which is then sent to the QueueManager to inform the QueueManager that the query has been completed. The query is then removed from the control node's internal lists.

7.3.1.4.4 SEARCH_FAILED

This message is received from a search node if there is some problem in the actual processing of the query on that node. If this message is received the control node first updates the search node metadata to indicate that the search node in question is no longer busy. It then sends creates a QUERY_FAILED message which is sent to the QueueManager to indicate that there is a problem with the query and that it couldn't be completed. The body of the QUERY_FAILED message will be a copy of the body of the SEARCH_FAILED message indicating the nature of the problem that caused the query to fail.

If there are other nodes still working on the query, no more processing is done at this time. If this was the last (or only) node working on the query the control node removes the query from its internal lists.

7.3.1.4.5 DISCONNECT

This message is received from many different sources and the exact effect this message has depends on the source. In all cases, the control node closes the socket and removes the connection to the program that sent the message from the list of open connections. When this message received from the Ingest program, nothing else happens. When received from the QueueManager, the control node sets a flag that the QueueManager is disconnected and all messages that would normally be sent to the QueueManager are not transmitted. If at some point the QueueManager reconnects, the flag will be unset and messages will again begin to flow.

When received from one of the search nodes, the action take depends on what the search node was doing. If the search node was idle, then the control node simply removes that node from its list of connected search nodes and closes the socket. If the search node was processing a query (i.e. a NEW_SEARCH message had been sent but a SEARCH_FINISHED had not been received), then the control node has to assume that that portion of the query was lost.

In this case it will find the next available search node that is idle and send the NEW_SEARCH message that was sent to the node that disconnected to the new node so that the query can be completed. If no nodes are currently available, the control node will set a flag indicating that there is a query that needs to be recovered and the first available search node will be used to process the query.

7.3.1.4.6 NEW_DATA_AVAILABLE

(Implemented and needs to be described)

7.3.1.4.7 REPROCESSED_DATA_AVAILABLE

This message is not used.

7.3.1.4.8 PROGRAM_CONTROL

The effects of this message depend on the content of the message body. The effects of the various commands that are received in the PROGRAM_CONTROL message are described in the following subsections.

7.3.1.4.8.1 Shutdown

The receipt of this command is a signal to shut down the entire system. When the SHUTDOWN command is received, the control node starts by closing its listen port so that no new connections can be established. If there are any idle search nodes, the control node forwards the message to them. It should receive a DISCONNECT message in response.

If there are any search nodes still processing queries, the control node waits for the receipt of the SEARCH_FINISHED message. When it receives this message, the SHUTDOWN message is sent to the search node. As queries that are currently being processed are completed the appropriate messages are sent to the QueueManager. Once all of the search nodes have been disconnected from the control node, the control node sends a DISCONNECT message to the QueueManager, closes the socket connection to the QueueManager, stores any state information needed (TBD) and shuts down.

7.3.1.4.8.2 Status

(Note: This message has never been implemented.) The receipt of this message is a request to know the current state of the data system. When the STATUS command is received, the control node will compose a STATUS_RESPONSE message from its internal metadata to send back to the QueueManager. The body of this message will contain the information described in Table 7.

Table 7 - Contents of the control node's STATUS_RESPONSE message

Body line number	Contents
1	Text string identifying database – ‘photon’, ‘event’ or ‘spacecraft’
2	Date & time stamp of server start up
3	Total time alive in HH:MM::SS format
4	Number of queries completed in ‘%d %d %d %d’ format where the first number is the total and the other three are the counts for the individual priority queues.
5	Average real time (in seconds) per query in “%f %f %f” format, one per queue
6	Average total processing time (in seconds) per query in “%f %f %f” format, one per queue. The total processing time is the sum of the time required by each search node involved to actually process the query.
7	Average number of nodes used per query in “%f %f %f” format, one per queue
8	Number of currently open queries in ‘%d %d %d %d’ format where the first number is the total and the other three are the counts for the individual priority queues.
9	Text string “Processing Queries”
10 to n	Query ID strings of queries currently being processed, one query ID per line
n+1	Text string “Search node status”
n+2 to m	Status of the connected search nodes, one node per line. This line will contain the following information, separated by pipe ‘ ’ characters: • Node name • Date & time stamp of node connection

	• Total time connected, in seconds • Total time spend processing queries, in seconds • Number of queries processed • Query ID string of query currently processing if any, otherwise text string 'idle'
--	--

7.3.2 Search Node

For a typical search, this is where most of the work is done. Unlike the QueueManager and the Control node, the search node process is not an asynchronous message handler but instead has a single processing path that it passes through for each query processed. The search node is a TCP/IP client to the control node and connects on start up.

The basic program flow for this process is shown in Figure 9 below. Like the control node, the search node starts by reading in the configuration data from the configuration file and then connecting to the MySQL database to get the file metadata for the internal data files. Once this data is read in the node verifies that its internal files match the data from the database. This is necessary as there may have been a data ingest while the node was down. If the search node's data files are out of sync, the node starts by updating its internal files. Next, the search node connects to the control node and provides its name via the IDENTIFY message. At this point the node just sits and waits for a message from the control node.

If the message is a NEW_SEARCH message, the node will read in the node identifier, the row filter command and the list of files to search. It will create an output file with a name based on the query ID and the node identifier and then loop over all the input files and applying the row filter to the data in files. The events that make the cut will be written to the output file. Once all the input files have been processed, the output file is closed and moved to the FTP site. Once there, the search node will send a SEARCH_FINISHED message back to the control node and wait for another message.

If the message is a PROGRAM_CONTROL:SHUTDOWN message, the search node will simply send a DISCONNECT message back to the control node, close the socket connection and shut itself down.

If the message is a NEW_ or REPROCESSED_DATA_AVAILABLE message, the search node will copy the new files from the control node's central repository and update its internal metadata tables to reflect the change. It will then send a DATA_ADDITION_COMPLETE message and wait for another message.

Database Search Node

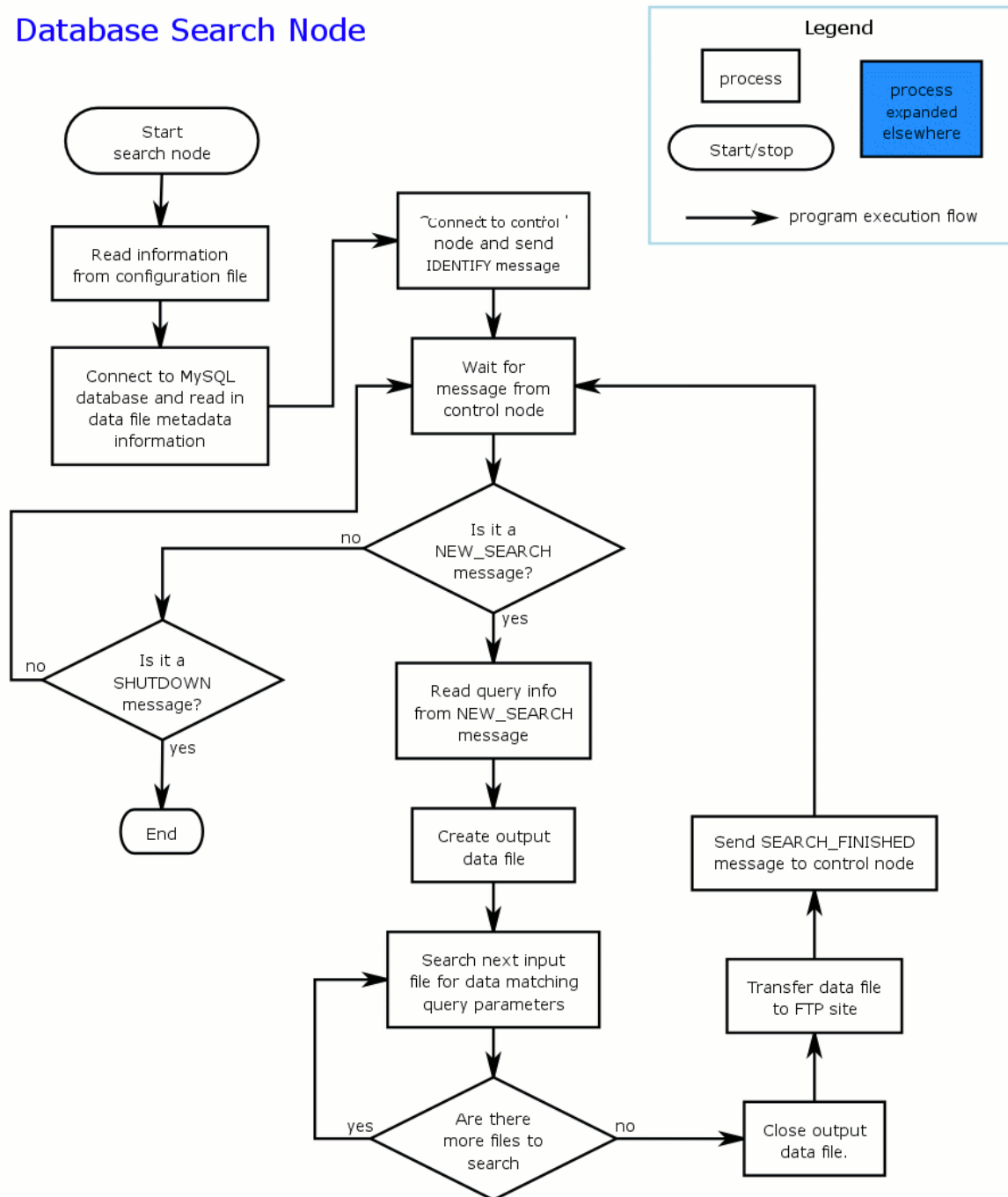


Figure 9 - Basic program flow for the database search node process.

7.4 Ingest System

The role of the ingest system is to convert the raw data files received from the LISOC into the internal data file format used by the data servers. The system must be able to handle both new data received from the instrument as well as reprocessed old data and update the data tables accordingly.

7.4.1 Program Flow

The flow of the ingest program depends on whether or not the system is working on new or reprocessed data files.

7.4.1.1 New Data

For new data the process is fairly straight forward. The program first checks to see if the specified data file has already been added to the system. If it has, an error is generated and the program exits. If not, it then reads in the time frame covered by the data in the new file and determines which files need to be modified. This is done by comparing the start time in the new data to the times in the most recent internal data files and the configuration file entry governing the size of the data time bins. If the current data file has reached its maximum size, a new set data files is started using the data from the input file. If not, the data from the input file are appended to the data in the existing files.

Once the new or updated data files have been created, the ingest program connects to the database server and sends and NEW_DATA_AVAILABLE message listing all of the new files that are available. It then waits for a response from the data server. If a DATA_ADDITION_COMPLETE message is received, the ingest program updates the MySQL data tables with the new metadata for the current files and then exits. If a DATA_ADDITION_ERROR message is received, the ingest program exits with an error.

7.4.1.2 Reprocessed Data

The handling of reprocessed data is a little more involved. *This has been implemented and needs to be detailed.*

8 MySQL Database Table Descriptions

This section describes the MySQL database tables used by the system. *Not all the tables are currently described in this section.*

8.1 Ingest_data Table

- db_name – The name of the database (photon, event of spacecraft)
- start_time – The start time in MET of the current set of region files for new ingest
- current_file_version – The current version number for the current set of region ingest files
- DB_version – The overall database version number. (Incremented after ingest of reprocessed data)
- allsky_start_time – The start time in MET of the current allsky file for new ingest
- current_allsky_version – The current version number for the current allsky ingest file
- data_start – The time, in MET, of the start of the data in the system
- data_end – The time, in MET, of the current end of data in the system.
- Yearly_start_time - The time, in MET of the start of they yearly spacecraft files. This column is no longer used
- timePerEvent - The average number of seconds it takes for the data server to search on a single event. This is used, along with the latency value below to estimate query times. Values are based on searches on a single node.
- latency - The minimum time to do search. This value serves as a zero point for the calucation of the estimated search time (i.e. time = latency + timePerEvent * number of events)

Field	Type	Null	Key	Default	Extra
db_name	varchar(20)		PRI		
start_time	double	yes		NULL	
current_file_version	int(11)	yes		NULL	
DB_version	int(11)	yes		NULL	
allsky_start_time	double	yes		NULL	
current_allsky_version	int(11)	yes		NULL	
data_start	double	yes		NULL	
data_end	double	yes		NULL	
yearly_start_time	double	yes		NULL	
timePerEvent	float	yes		NULL	
latency	float	yes		NULL	

8.2 photon_file_comp Table

- file_base – The root filename of the set of region files that the data was added to
- version - The database version at time of first ingest (not used)

- run - The run number the ingested file corresponds to.
- run_version - The version number of the most recently ingest file for this run.
- date_added – The date and time the data was added to the system.
- nPhot - The number of photon events in the ingested file.
- tStart - The TSTART value of the ingested file.
- tStop - The TSTOP value of the ingested file.

Field	Type	Null	Key	Default	Extra
file_base	varchar(30)	yes		NULL	
version	tinyint(4)	yes		NULL	
run	varchar(40)	NO	PRI		
version	int(11)	NO		0	
date_added	datetime	yes		NULL	
nPhot	int(11)	yes		NULL	
tStart	double	yes		NULL	
tStop	double	yes		NULL	

8.3 *photon_file_data Table*

- filename – The full filename of the internal data file
- modified – The data and time the file was last modified
- n_photons – The total number of photons/events stored in the file
- start_time – The time, in MET, of the start of data in the file
- stop_time – The time, in MET, of the end of data in the file
- first_DB_version – The first database version this file applies to
- last_DB_version – The last database version this file applies to
- HTM_pixel – The HTM pixel that the data in this file covers

Field	Type	Null	Key	Default	Extra
filename	varchar(40)		PRI		
modified	datetime	yes		NULL	
n_photons	int(11)	yes		NULL	
start_time	double	yes		NULL	
stop_time	double	yes		NULL	
first_DB_version	int(11)	yes		NULL	
last_DB_version	int(11)	yes		NULL	
HTM_pixel	varchar(5)	yes		NULL	

8.4 *photon_allsky_file_comp Table*

- file_base – The root filename of the allsky file that the data was added to
- component – The name of the input file the data was extracted from
- date_added – The date and time the data was added to the system.

Field	Type	Null	Key	Default	Extra
file_base	varchar(30)	yes		NULL	
component	varchar(40)		PRI		
date_added	datetime	yes		NULL	

8.5 *photon_allsky_file_data Table*

- filename – The full filename of the internal data file
- modified – The data and time the file was last modified
- n_photons – The total number of photons/events stored in the file
- start_time – The time, in MET, of the start of data in the file
- stop_time – The time, in MET, of the end of data in the file
- first_DB_version – The first database version this file applies to
- last_DB_version – The last database version this file applies to

Field	Type	Null	Key	Default	Extra
filename	varchar(40)		PRI		
modified	datetime	yes		NULL	
n_photons	int(11)	yes		NULL	
start_time	double	yes		NULL	
stop_time	double	yes		NULL	
first_DB_version	int(11)	yes		NULL	
last_DB_version	int(11)	yes		NULL	

8.6 *query_params Table*

- ID – Internally assigned id value
- queryID - The query ID assigned by the web interface
- source - The source of the query (not currently used).
- received - The date and time the query was received
- completed - The date and time the query was completed
- email - the e-mail address to send completion notifications to
- photon - Flag for whether or not the query should go to the photon server
- event - Flag for whether or not the query should go to the event server
- spacecraft - Flag for whether or not the query should go to the spacecraft server
- RA – The Right Ascension of the query center
- Decl – The Declination of the query center
- shape – The shape of the query region (currently only circle allowed)
- r1 – The radius of the circular region (also the RA range for box searches)
- r2 – The Dec range for box searches
- angle – The rotation angle for box searches
- tStart – The start time for the search
- tStop – The end time for the search
- eMin – The lower energy limit for the search

- eMax – The upper energy limit for the search
- comment – Comment field
- notified - Flag for whether or not a message has been sent to the specified email address if there is one.

Field	Type	Null	Key	Default	Extra
ID	int(11)	NO	PRI	NULL	auto_increment
queryID	varchar(40)	NO	MUL		
source	varchar(40)	YES		NULL	
received	datetime	YES		NULL	
completed	datetime	YES		NULL	
email	varchar(100)	YES		NULL	
photon	tinyint(1)	YES		0	
event	tinyint(1)	YES		0	
spacecraft	tinyint(1)	YES		0	
RA	float	yes		NULL	
Decl	float	yes		NULL	
shape	varchar(10)	yes		NULL	
r1	float	yes		0	
r2	float	yes		0	
angle	float	yes		0	
tStart	double	yes		NULL	
tStop	double	yes		NULL	
eMin	float	yes		NULL	
eMax	float	yes		NULL	
comment	varchar(160)	yes		NULL	
notified	tinyint(1)	yes		0	

8.7 *photon_query_status Table*

- ID - internal id assigned by the database
- query – The ID value of the query from the query_params table
- started – The date and time the query processing began
- completed – The date and time the query processing ended
- elapsed – The total amount of time spent working on the query
- events_searched – The total number of events searched through to match the query
- events_returned – The total number of events returned by the query
- n_files – The total number of files the query results were returned in
- queue – The priority queue the query was assigned to

Field	Type	Null	Key	Default	Extra
ID	int(11)	NO	PRI	NULL	auto_increment
query	int(11)	NO	UNI	NULL	
started	datetime	yes		NULL	
completed	datetime	yes		NULL	

elapsed	double	yes	NULL
events_searched	bigint(20)	yes	NULL
events_returned	bigint(20)	yes	NULL
n_files	smallint(6)	yes	NULL
queue	int(11)	yes	NULL
